

Interazione Uomo Macchina

Interfacce

Def. Un'interfaccia è mezzo attraverso cui l'interazione o la comunicazione avviene.

Più nello specifico, nel contesto proprio dell'Interazione Uomo-Macchina (HCI), è fondamentale sottolineare che l'interfaccia è il confine (boundary) e il ponte tra due entità:

- **L'essere umano:** Parla il linguaggio degli **obiettivi** e delle **intenzioni** (es. "Voglio un caffè.").
- **La macchina:** Parla il linguaggio dei **comandi** e degli **stati interni**.

Def. L'interfaccia è l'elemento che traduce le intenzioni umane in istruzioni che la macchina può eseguire, e lo stato della macchina in un *feedback* che l'uomo può comprendere.

Esempio: Nel caso di un computer, anche la tastiera o il mouse possono rappresentare un'interfaccia. Nel caso dello smartphone invece l'interfaccia principale è lo schermo. Al di fuori della sfera tecnologica, ad esempio nel caso delle automobili, un'interfaccia può essere rappresentata dal volante, dal cambio, dai pedali, cioè tutti quegli elementi che ci permettono di impartire comandi alla macchina. Ma anche il tachimetro, le spie, gli specchietti, poiché permettono all'auto di interagire con noi (comunicare qualcosa).

Tutte queste interfacce sono state progettate in modo specifico per garantire determinate prestazioni. Tuttavia, nulla vieta di riprogettarle per adattarle a esigenze particolari (ad esempio, macchine per persone con disabilità).

User Interface - UI

L'interfaccia utente, nell'ambito dell'interazione uomo-macchina, è lo spazio che consente all'essere umano di controllare ed utilizzare efficacemente (tramite input) la macchina, la quale, contemporaneamente, restituisce informazioni (output) che aiutano l'utente nei processi decisionali.

Spesso infatti, le interazioni tra uomo e macchina avvengono in canali di comunicazione bidirezionali con componenti sia **hardware (fisiche)** che **software (logiche)**. Ad esempio, lo schermo fisico di un telefono costituisce la parte hardware dell'interfaccia, mentre ciò che viene visualizzato al suo interno rappresenta la parte software.

Human Computer Interaction – HCI

L'**Interazione Uomo-Macchina** consiste nella **progettazione**, **valutazione** e **implementazione** di sistemi interattivi per l'uso umano. È un campo di studio Multidisciplinare poiché, a seconda degli ambiti, può riguardare una grande quantità di scienze diverse (Es. tecnologia ed ergonomia si fondono insieme per realizzare mouse, tastiere, display, etc...)

Essa coinvolge due entità principali: **l'essere umano** e **il computer**, che determinano reciprocamente il proprio comportamento nel tempo. L'analisi viene inquadrata in termini di **obiettivi umani** e dei relativi task necessarie per perseguirli.

Come progettare e sviluppare Buoni Sistemi Interattivi?

Un buon processo di progettazione deve essere **iterativo** e **centrato sull'utente**, basato su:

- bisogni delle persone (what they NEED, not what they WANT);
- principi e linee guida di design;
- obiettivi di usabilità;
- prototipazione rapida;
- varie forme di valutazione;
- programmazione.

Gli errori **non devono mai essere considerati colpa dell'utente**, ma conseguenza di un cattivo design. Gli esseri umani sono per natura imprecisi, distratti e non-onniscienti... Pertanto il sistema deve:

- anticipare questi comportamenti;
- minimizzare la probabilità di azioni inappropriate (fase di valutazione);
- massimizzare la possibilità di individuare e correggere un errore (fase di esecuzione);
- permettere all'utente di comprendere lo stato del sistema e costruire un modello mentale adeguato.

Usabilità - Definizione ISO

Def. L'usabilità descrive "quanto bene un sistema, o servizio, può essere usato dagli utenti per raggiungere specifici obiettivi con **efficacia**, **efficienza** e **soddisfazione** in uno specifico contesto d'uso".

In particolare l'usabilità è:

- **goal-dependent** poiché una stessa interfaccia può risultare più o meno usabile a seconda dell'obiettivo che si vuole raggiungere. *Es. la Command Line è usabile da un informatico per spostare, creare o cancellare file, ma meno per scaricare documenti dal web.*
- **user-dependent** poiché non tutti gli utenti sono in grado di utilizzare una stessa interfaccia. *Es. la Command Line è facilmente utilizzabile da studenti di informatica, ma meno da studenti di altri corsi.*

E possiamo valutarla velocemente considerando:

- **Utilità:** fa davvero qualcosa che le persone desiderano?
- **Apprendibilità:** è facile da imparare?
- **Memorabilità:** una volta imparato, è facile da ricordare?
- **Efficacia:** consente di raggiungere l'obiettivo?
- **Efficienza:** una volta appreso, è veloce da usare?
- **Visibilità:** lo stato del sistema è visibile?
- **Errori:** gli errori sono pochi e recuperabili?
- **Soddisfazione:** è piacevole da usare?

Design Processes and Frameworks

Def I processi di design sono metodologie che guidano la progettazione e lo sviluppo di sistemi interattivi.

Ne esistono molti tipi poiché ogni contesto e progetto ha esigenze, vincoli e obiettivi diversi. Non può esistere un unico processo che funzioni bene in ogni situazione.

User Centered Design - UCD

Il **Design Centrato sull'Utente** nasce per ridurre il rischio di fallimento dei progetti software (si stima che circa il 50% dei progetti fallisca a causa di scarsa comunicazione tra sviluppatori e utenti finali). Principi:

- Tenere conto dei **bisogni, limiti e capacità degli utenti finali reali** in ogni fase del processo di design.
- Scoprire i problemi nelle **prime fasi**, così da ridurre i cambiamenti necessari ogni volta

PRO:

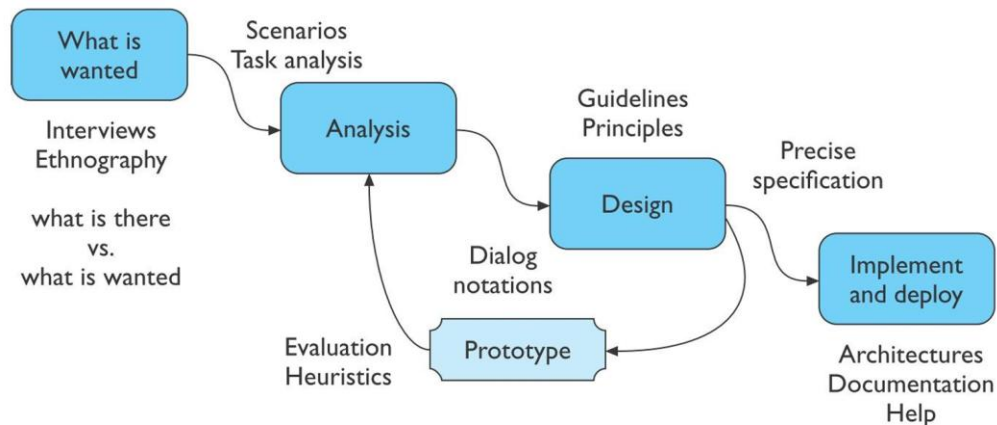
Sistemi più facili da imparare.
Prestazioni più veloci.
Meno errori umani.
Stimola la scoperta di funzionalità avanzate.
Evita di costruire "il sistema sbagliato".

CONTRO:

Come trovare gli utenti giusti?
Quanti ne servono?
Come parlare la loro lingua?
Come estrarre i bisogni reali (utenti, business, organizzazione)?

Nota: Il problema principale è che non possiamo andare semplicemente dall'utente e chiedere "di cosa hai bisogno? Cosa vorresti nel progetto?" perché probabilmente la sua risposta sarebbe condizionata. Difficilmente riuscirebbe a fornirci informazioni utili di primo impatto.

Fasi del Processo UCD



1. **Needfinding:** Individuare i bisogni dell'utente. Cosa è desiderato? Cosa è veramente necessario? Le tecniche principali sono: osservazione, interviste, shadowing.
2. **Analisi:** Formalizzare e strutturare i bisogni raccolti. Confrontare situazione attuale e nuova attesa.
3. **Design:** Prendere le decisioni principali che danno forma al sistema, applicando regole, linee guida e principi di design. Considerare tipologie diverse di utenti, capire come deve avvenire l'interazione. Progettare il layout visivo.
4. **Prototipazione e Valutazione:** Il design deve essere verificato in modo incrementale, un'iterazione alla volta. Sarà fondamentale coinvolgere gli utenti in ogni ciclo.
5. **Implementazione e Documentazione**

Participatory Design

Un passo oltre l'UCD: **gli utenti partecipano attivamente alla progettazione** dei sistemi e delle applicazioni che utilizzeranno. L'obiettivo principale prevede infatti di coinvolgere attivamente gruppi di utenti reali nelle discussioni e nella realizzazione delle bozze del progetto finale.

Tuttavia richiede moderatori con competenze forti per interfacciarsi con gli utenti, che non sono "professionisti nel settore". Si ha quindi bisogno di filtrare idee, premiare la partecipazione, gestire resistenze. Più efficace con utenti maturi e con un minimo di conoscenza del dominio.

Agile Interaction Design

Questo approccio prende spunto dall'Agile Development del software engineering. Si basa su uno **Sviluppo Evolutivo**: il sistema viene costruito incrementando versioni in cicli rapidi di rilascio. L'obiettivo principale è quindi ridurre al minimo i tempi tra idea, realizzazione e verifica con gli utenti.

Tuttavia si rischia di incappare in analisi troppo superficiali, focus eccessivo sulla velocità e poco coinvolgimento degli utenti. Dunque, funziona bene solo se si accolgono gli utenti, i quali devono essere attivi e disponibili, fornendo feedback rapidi ma efficaci.

Needfinding

Nel nostro corso ci concentreremo principalmente sull'UCD (design centrato sull'utente). Un aspetto fondamentale dell'UCD è comprendere i bisogni reali degli utenti. Tuttavia, non possiamo andare da loro e chiedere direttamente "Di cosa avresti bisogno?". Gli utenti, infatti, non sempre sono consapevoli delle proprie esigenze e spesso non hanno il linguaggio o le competenze per esprimerle in modo utile.

Per questo motivo nasce il concetto di **Needfinding**: un insieme di metodi e pratiche che permettono di scoprire i bisogni (espliciti e impliciti) degli utenti, senza doverli chiedere direttamente, ma osservando e interpretando i loro comportamenti, abitudini e segnali.

Esempio: Consideriamo la figura qui sotto:



Nessuno ha esplicitamente detto "vorrei un passaggio diretto". Tuttavia, osservando il comportamento degli utenti (che tracciano un percorso sull'erba), si intuisce chiaramente il bisogno: abbreviare il percorso.

Questo è un esempio concreto di Needfinding: capire cosa serve osservando i segnali reali delle persone.

Tra le Tecniche Principali di Needfinding troviamo:

- **Osservazione:** consiste nell'osservare direttamente le persone nel loro contesto naturale, studiando comportamenti, interazioni e pratiche quotidiane. L'approccio etnografico permette di cogliere bisogni latenti che gli utenti magari non esprimerebbero a parole.
- **Diaries:** si chiede alle persone di registrare le proprie attività, pensieri, emozioni o difficoltà in un diario. Questo metodo raccoglie dati longitudinali e permette di capire abitudini e bisogni nel tempo.
- **Interviews:** conversazioni strutturate o semi-strutturate con gli utenti, per esplorare motivazioni, esperienze e bisogni. Consentono un approfondimento qualitativo e flessibile.
- **Focus groups:** discussioni guidate con piccoli gruppi di utenti, utili per far emergere idee, opinioni e dinamiche sociali, e per confrontare diversi punti di vista in un'unica sessione.
- **Questionari:** questionari distribuiti a un campione ampio di persone, utili per raccogliere dati quantitativi e individuare pattern o tendenze nei bisogni. Sono molto veloci, tante risposte raccolte in poco tempo. Possono essere analizzati più rigorosamente. Tuttavia sono meno flessibili delle interviste. Infatti, l'assenza di un umano a condurre ed il fatto di dover scegliere a priori le domande, ci impediscono di navigare più a fondo le risposte degli utenti. Pertanto sarà fondamentale pensare prima a come saranno analizzate le domande, cosa vogliamo ottenere. Altrimenti rischiamo di ritrovarci con risposte inutili, che non ci serve analizzare.
- **Indagine Contestuale:** tecnica ibrida di intervista e osservazione. L'utente svolge le proprie attività mentre il ricercatore osserva, fa domande e cerca di comprendere processi, difficoltà e obiettivi.
- **Camera Studies:** fornire agli utenti una videocamera per riprendere la propria attività e permettere all'analista di "vivere con gli occhi dell'utente"

A prescindere da quale tecnica si deciderà di utilizzare il consiglio fondamentale è: Concentrarsi su focus specifici di utenti. Anche quando le applicazioni hanno target molto ampi è sempre possibile dividere gli utenti in categorie ed analizzare separatamente ognuno di queste.

Esempio: Whatsapp è usato praticamente da tutti, ma si possono dividere gli utenti per categorie. Ad esempio si possono intervistare separatamente gli anziani, i giovani istruiti, quelli meno istruiti, i lavoratori, i non lavoratori, etc...

Osservazione

Osservazioni in Ambiente Controllato: Ad esempio in un laboratorio. Non sto andando nell'ambiente dell'utente ma è lui che viene nel mio. Questo ci potrebbe permettere analisi più rapide ed efficienti, senza distrazioni o interruzioni provenienti dall'esterno (che potrebbero verificarsi se invece fossimo in ambienti non controllati). Tuttavia l'ambiente controllato potrebbe causare un **Hawthorne Effect**, per cui l'utente potrebbe sentirsi osservato, sotto pressione, e quindi performare male o diversamente da quello che farebbe normalmente, "sporcando" i dati in analisi.

Osservazioni in Ambiente NON Controllato: Anche detto "In the Wild". Consiste nell'osservare l'utente mentre utilizza un prodotto nella vita reale, senza un contesto strutturato o controllato come un laboratorio. Questo garantisce maggiore affidabilità, evita l'Hawthorne Effect e risulta utile nella generazione di idee per migliorare il prodotto, offrendo prospettive che non emergono dai soli dati quantitativi. Tuttavia risulta difficile ottenere un campione rappresentativo: L'osservazione naturalistica è spesso più costosa e richiede molto tempo, quindi il numero di partecipanti è limitato. Inoltre, poiché si osserva nel mondo reale, si è soggetti a fattori ambientali (come meteo o rumore) che possono influenzare il comportamento degli utenti.

Interviste

Poiché c'è una conduzione umana ci permettono di approfondire, ponendo delle contro-domande che esplorano il comportamento dell'utente, oppure scoprire informazioni inaspettate (es. la risposta dell'utente ci fa spostare il focus dell'intervista su un aspetto del contesto che non avevamo considerato).

Per ottenere una buona intervista bisogna innanzitutto capire chi intervistare. Come già detto in precedenza, in tutte le tecniche di needfinding, si punta a persone rappresentative degli utenti, divisi in categorie (o target) specifici. Si può anche decidere di intervistare persone esperte di sistemi simili al nostro, così che abbiano già qualche tipo di esperienza, o anche persone che non sono potenziali utenti, cioè persone che non centrano nulla. Questo potrebbe aiutarci a capire come far entrare nuovi utenti nel sistema o trovare need cui nessuno "all'interno" avrebbe pensato. Bisogna poi preparare un buon Set di Domande. Sono da evitare le domande:

- la cui risposta è scontata
- che contengono la risposta
- troppo generiche
- che chiedono all'utente di sostituirsi al designer
- basate su scenari ipotetici
- che chiedono "quanto spesso" o "ogni quanto" fai qualcosa
- che prevedono come risposta sì/no
- che cominciano con "ti piacerebbe...?" o "useresti...?"

Si consigliano invece:

- domande che prevedono risposte aperte
- andare sul concreto, essere specifici
- focalizzarsi su situazioni note all'intervistato
- chiedere quando (ma non lontano nel tempo)

Interviste Specifiche

Nel contesto delle **interviste agli utenti**, è importante considerare **diverse categorie di intervistati**, ciascuna capace di offrire **tipi di informazioni differenti**. Saper gestire e interpretare correttamente questi contributi è fondamentale per ottenere risultati utili alla progettazione:

- **Lead Users:** Sono particolarmente competenti e sofisticati. Spesso sviluppano da soli soluzioni ai propri bisogni. Ci indicano i need che molti avranno nel prossimo futuro
- **Extreme Users:** Spingono un sistema al limite. Permettono di scoprire problemi nascosti o difficili da individuare. I loro bisogni riflettono bisogni comuni, ma in forma più evidente. Sono preziosi perché le loro esigenze (di solito alla robustezza o accessibilità) rivelano problemi nascosti che, una volta risolti, migliorano l'esperienza per l'utente medio.
- **Expert Users:** Hanno sviluppato un Modello Mentale robusto e preciso del sistema. Sono utenti ad alta frequenza e lunga durata d'uso. Cercano la massima efficienza e usano estensivamente scorciatoie (shortcut), comandi da tastiera, e personalizzazione. Sono cruciali per i test di efficienza pura e per capire dove il design rallenta l'utente che conosce già il sistema.

Interviste particolari

Queste metodologie qualitative permettono di andare oltre il semplice "cosa" e di scoprire il "perché" e il "come" del comportamento dell'utente:

- **interviste a esperti:** Non si intervista per capire un singolo utente, ma per ottenere una visione aggregata del comportamento di molti utenti. Si consigliano soprattutto all'inizio del progetto, quando si necessita rapidamente di una panoramica di alto livello o quando l'accesso agli utenti finali è difficile. Tuttavia, è bene ricordare che l'esperto può essere soggetto al bias dell'esperto (dimentica le difficoltà del novizio).
- **history interviews:** Si chiede all'intervistato di raccontare una storia specifica che ha portato a una situazione attuale, concentrandosi sulla cronologia degli eventi e sulle decisioni prese in quel momento. Aiuta a comprendere le cause profonde di un comportamento o di un problema tracciandone la sequenza temporale.
- **process mapping:** Si chiede all'intervistato di descrivere meticolosamente ogni singola azione, decisione o strumento che utilizza per completare un compito (es. "dimmi esattamente cosa fai per inviare una fattura"). Si ottiene una visione step-by-step del flusso di lavoro (il *task flow*) dell'utente. Permette di identificare pain points (punti di frizione), task cruciali e deviazioni non previste dal processo ideale. È essenziale per la successiva Task Analysis.
- **laddering:** Indagare sempre più a fondo. Chiedere sempre "perché? perché? ...?" . Serve a svelare la motivazione profonda e i valori che guidano il comportamento
- **intercepts:** Interviste molto brevi, basate su una sola domanda, utili per raccogliere feedback immediati o impressioni "a caldo".

Task Analysis

Def Con Task Analysis intendiamo lo studio del modo in cui le persone svolgono le loro attività. L'obiettivo è determinare: cosa fanno, cose usano e quanto bene riescono (obiettivi).

Need, Task e Goal

Goal è qualcosa che vogliamo raggiungere.

Need è quello di cui ho bisogno.

Task è la sequenza concreta di azioni che dobbiamo seguire.

Nel nostro corso semplifichiamo ed assumeremo che Need e Goal siano la stessa cosa.

Esempio:

- Goal: Superare l'esame
- Need: Acquisire le conoscenze necessarie
- Task: Prendere appunti, studiare, esercitarsi

Storyboard

Lo storyboard è una rappresentazione grafica dell'aspetto esterno di un sistema, senza funzionalità reali. È paragonabile a un fumetto disegnato a mano che mostra l'esecuzione di un compito.



Serve per:

- Raccontare visivamente come una persona interagisce con il sistema per raggiungere un obiettivo.
- Comunicare il flusso temporale delle azioni e delle interazioni principali.
- Non richiede abilità artistiche: ciò che conta è comunicare l'idea, non fare disegni "belli".

Ogni storyboard:

- Deve **includere sempre le persone**, mai le interfacce.
- Mostra **come si svolge un compito passo dopo passo**, fino al raggiungimento di un risultato soddisfacente. Poco contesto (massimo una vignetta iniziale e una finale) e tanta descrizione dei passi.
- È focalizzato sul **task** dell'utente.

Cosa deve contenere uno storyboard

1. **Setting (contesto):** Chi sono le persone coinvolte. In che ambiente si trovano. Quale compito stanno svolgendo. Cosa li spinge a farlo. (massimo una vignetta iniziale ed una finale)
2. **Sequence (sequenza):** Le varie azioni o passaggi necessari. Il ruolo del sistema nel supportare l'utente.
3. **Satisfaction (soddisfazione):** Qual è la motivazione dell'utente. Qual è il risultato finale e quale bisogno viene soddisfatto

ATTENZIONE! Lo storyboard NON deve contenere le schermate/interfacce dell'applicazione. Quelle non le abbiamo ancora progettate.

Tipi di storyboard

Tradizionale: Usa convenzioni da fumetto (personaggi, nuvolette di dialogo, sfondi). Può includere note esplicative sotto ogni scena.

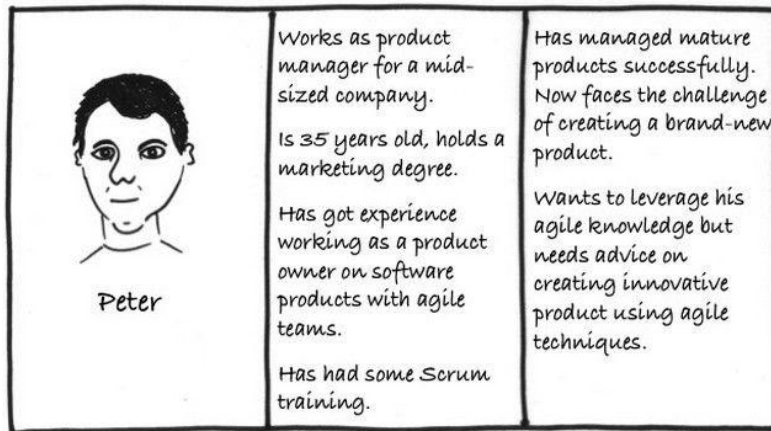
Scored storyboard: Usato quando l'interfaccia è dinamica o include media (movimento, suoni, colori). Include annotazioni specifiche su questi elementi.

Text-only storyboard: Usato quando il comportamento dell'interazione è troppo complesso per essere rappresentato graficamente. Consiste in una descrizione testuale più lunga e dettagliata.

Personas

Def. Le personas sono archetipi fittizi, creati con l'obiettivo di rappresentare i bisogni, gli obiettivi, i comportamenti e le frustrazioni di un target specifico di utenti.

Non sono utenti reali, ma profili sintetici basati su dati di ricerca reali (interviste, osservazioni, sondaggi, ecc.) raccolti durante la fase di Needfinding e Analisi.



Rappresentano uno strumento fondamentale nel design di interfacce e, più in generale, nel processo di progettazione centrato sull'utente (User-Centered Design). Rappresentano modelli astratti di utenti reali, costruiti per aiutare i designer a comprendere meglio esigenze, comportamenti e obiettivi delle persone che utilizzeranno il sistema.

Si costruisce a partire da dati concreti, raccolti tramite ricerca sugli utenti:

- **dati demografici** (età, sesso, professione, istruzione, ecc.)
- **dati comportamentali** (come interagisce con il sistema, abitudini, frequenza d'uso)
- **dati psicografici** (valori, motivazioni, atteggiamenti, interessi)

NOTA: Le personas non rappresentano singoli individui reali, ma categorie di utenti con caratteristiche, bisogni e obiettivi comuni.

Obiettivi principali delle Personas

- **Creare empatia:** i progettisti si immedesimano nell'utente, vedendo il mondo dal suo punto di vista.
- **Guidare le decisioni di design:** aiutano a scegliere soluzioni coerenti con i bisogni dell'utente.
- **Ottimizzare l'esperienza utente (UX):** offrono un quadro chiaro per capire dove intervenire.
- **Fornire un riferimento comune** nel team di progetto: facilitano la comunicazione tra designer, sviluppatori e stakeholder.

Elementi costitutivi di una Persona

Ogni persona dovrebbe contenere:

- **Nome:** dà identità e umanità alla scheda.
- **Foto:** favorisce la connessione empatica.
- **Caratteristiche demografiche:** età, sesso, professione, livello di istruzione.
- **Obiettivi:** cosa vuole ottenere con il sistema/interfaccia.
- **Frustrazioni:** problemi, limiti o difficoltà che sperimenta.
- **Comportamenti:** come utilizza il prodotto, con quale frequenza e in quali contesti.

Prototipi

Def Un prototipo è un modello di un sistema interattivo che simula o anima alcuni aspetti, caratteristiche e funzioni di un prodotto o servizio

La Prototipazione è una fase fondamentale del processo di progettazione centrato sull'utente (*User-Centered Design*). Prototipare significa **creare versioni preliminari** di un sistema interattivo, per esplorare idee, verificarne la fattibilità e testarle con gli utenti reali.

Serve per valutare specifiche caratteristiche, come il flusso di navigazione, la disposizione degli elementi o la reazione dell'utente, senza dover realizzare il sistema completo. Un prototipo realistico dovrebbe essere utilizzato in condizioni simili a quelle reali, in modo da ottenere feedback affidabili. È come se simulassimo solo il frontend, senza nessun backend funzionante.



In generale i prototipi hanno qualcosa che funziona davvero, è stato progettato e testato interamente. Altre cose sono parziali, simulate, in fase di test, mentre altre funzionalità non ci sono proprio.

Tecniche di Prototipazione

Esistono tre principali approcci:

1. **Throw-away Prototyping:** Letteralmente “Usa e Getta”. Il prototipo viene realizzato solo per esplorare un'idea o comprendere un problema, poi viene scartato. L'obiettivo è imparare, non costruire un prodotto finale.
2. **Incremental Prototyping:** Il prototipo non viene buttato, ma viene ampliato progressivamente, aggiungendo via via nuove funzionalità o miglioramenti.

Esempio: Immagina di dover costruire un grande edificio. Con l'Incrementale prima costruisci e completi un'ala (un set di funzionalità), poi ne costruisci un'altra e così via, fino a quando tutte le ali necessarie sono al loro posto.

3. **Evolutive Prototyping** Ogni prototipo serve come base per il successivo, che ne mantiene la struttura ma la perfeziona. È un approccio ciclico e cumulativo. Non si concentra solo sull'aggiungere nuove funzioni, ma sul continuo miglioramento e raffinamento dell'intera struttura esistente man mano che si ottengono feedback

Esempio: inizi con una tenda molto semplice (Prototipo 1), ottieni feedback, poi la rafforzi e la ingrandisci aggiungendo muri e fondamenta migliori (Prototipo 2), poi aggiungi l'impianto elettrico e idraulico, modificando la struttura di base ad ogni passo. Non stai solo aggiungendo, stai facendo evolvere tutto il sistema.

I progettisti non sono infallibili: il primo progetto non è quasi mai perfetto. Per questo motivo si adotta un ciclo iterativo di prototipazione, in cui ogni versione viene testata, valutata e migliorata. Un ciclo tipico è:

1. Progettare
2. Creare un prototipo (spesso evolutivo)
3. Validare (test con utenti o esperti)
4. Se i risultati sono insoddisfacenti, riprogettare

Ma lo vedremo meglio quando studieremo il metodo Agile

Best Practices

Per ottenere buoni risultati, è importante:

- **Capire cosa non funziona:** analizzare i problemi con metodo.
- **Formulare ipotesi di miglioramento:** non limitarsi a correggere sintomi ma risalire alle cause.
- **Partire da una base solida:** un buon primo prototipo accelera le iterazioni successive.
- **Prototipare solo quello che Serve:** Un errore comune è quello di prototipare tutto, quando in realtà ci basta fare solo ciò che è legato ai nostri task, risparmiando tempo.

Un errore comune è il cosiddetto “**hill climbing**”, ovvero migliorare localmente una soluzione senza rendersi conto che si è bloccati su un “punto alto” non ottimale. Serve quindi la capacità di rimettere in discussione decisioni iniziali, se necessario.

Problemi e limiti della prototipazione

Nonostante i vantaggi, il prototyping presenta alcune **criticità**.

- **Tempo:** Realizzare prototipi richiede tempo e risorse. Quando un prototipo viene scartato, può sembrare tempo sprecato. Tuttavia, il rapid prototyping può aiutare a velocizzare le fasi, MA NON bisogna accelerare la fase di valutazione. Quella è fondamentale che sia svolta con cura.
- **Pianificazione:** È difficile pianificare con precisione un processo di design iterativo. I costi sono difficili da stimare. Spesso è necessario negoziare tempi e risorse tra committente e progettista.
- **Caratteristiche non funzionali:** Aspetti come sicurezza, affidabilità o tempo di risposta sono spesso trascurati nei prototipi. È importante ricordare che questi elementi, pur non visibili nell’interfaccia, influenzano fortemente l’esperienza utente.

Envisionment

In questa lezione si approfondiscono le tecniche e gli strumenti di **envisionment**, ovvero la creazione di rappresentazioni tangibili delle idee di design, con diversi livelli di fedeltà e interattività. Obiettivi principali:

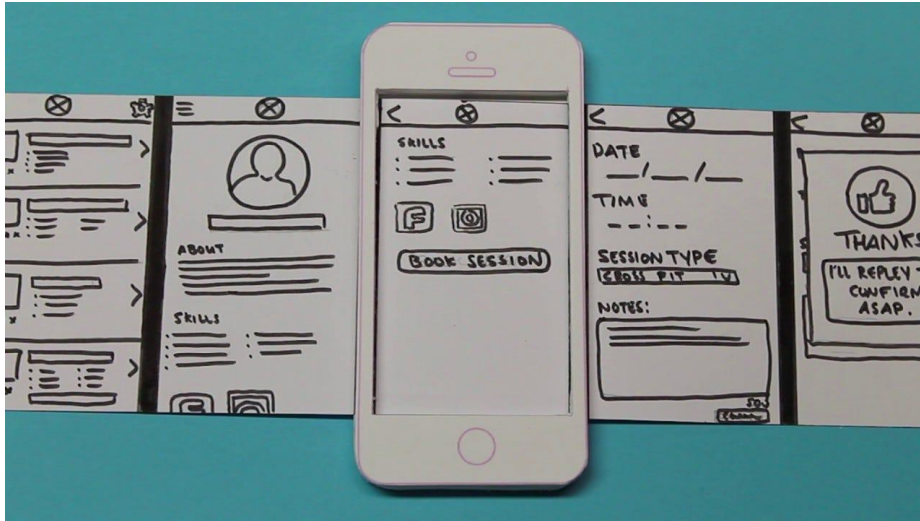
- **Generare nuove idee**, trasformando concetti astratti in rappresentazioni concrete.
- **Valutare le idee all’interno del team di design**, per discutere alternative e criticità.
- **Testare le idee con utenti reali**, raccogliendo feedback utili per la progettazione iterativa.

Le tecniche e gli strumenti utilizzati variano in base alla fase del design in cui ci troviamo (iniziale, intermedia, avanzata, finale) ed il pubblico di riferimento (designer, utenti di test, clienti, stakeholder, management).

Errore comune: concentrarsi prematuramente sull’interfaccia grafica (UI) senza aver prima compreso a fondo i compiti e gli obiettivi dell’utente.

Prototipi a Bassa Fedeltà

Def. i Low-Fidelity Prototypes sono rappresentazioni semplici e approssimative dell'interfaccia utente, realizzate spesso a mano su carta. Consentono di simulare l'uso di un'applicazione mesi prima dell'implementazione effettiva del software.



Esempio: Un *paper prototype* consiste in **mock-up disegnati a mano** dell'interfaccia utente, distribuiti su più fogli. Ogni foglio rappresenta una schermata o un componente dell'interfaccia. Durante i test, gli utenti interagiscono con i fogli come se fossero elementi digitali, mentre un membro del team simula il comportamento del sistema. Ad esempio:

- Indicare con il dito = clic del mouse
- Scrivere su carta = digitare
- Sostituire o aggiungere fogli = cambi di schermata o messaggi di sistema

Per questo motivo, test basati sui paper prototype, prevedono almeno 3 figure:

- **Computer Actor:** l'utente che simula il task.
- **Facilitatore:** colui che guida l'utente, presenta i task e incoraggia il *think aloud* (pensare ad alta voce).
- **Osservatore:** annota in silenzio comportamenti e problemi osservati.

In questo modo, il prototipo è a **bassa fedeltà visiva**, ma **ad alta fedeltà concettuale**, perché consente di testare il flusso logico e la struttura dell'interazione.

Per vedere un esempio completo consultare il video: <https://youtu.be/OlbdlXLunt4?t=596>

VANTAGGI:

- Velocità: disegnare è molto più rapido che programmare.
- Flessibilità: è facile modificare o sostituire elementi anche durante il test.
- Accessibilità: anche persone non tecniche possono contribuire alla progettazione.
- Nessun costo di sviluppo.

SVANTAGGI: Non si possono testare:

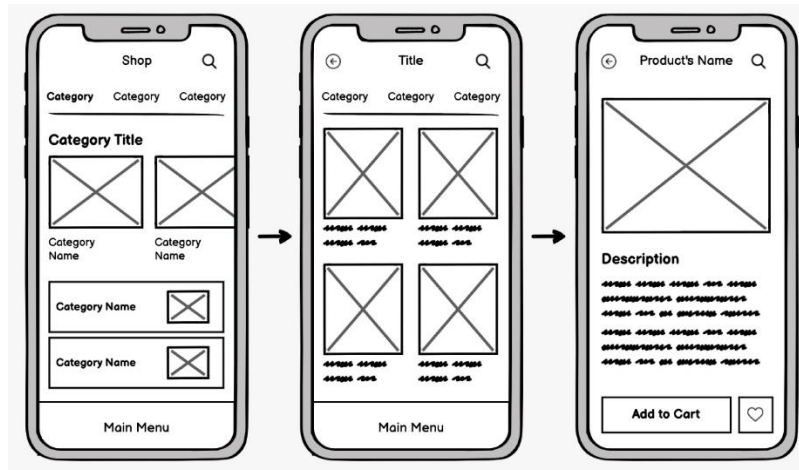
- Aspetti estetici (colori, font, spaziature)
- Prestazioni o tempi di risposta
- Efficienza dell'interazione
- Reazioni a piccoli cambiamenti (troppo evidenti su carta)

Prototipi a Media Fedeltà

Def i Medium-Fidelity Prototypes sono simulazioni digitali dell'interfaccia utente che mostrano l'aspetto visivo dell'applicazione e permettono una minima interattività, come il passaggio da una schermata all'altra. Sono spesso chiamati **mockup** o **wireframe interface**.

Caratteristiche:

- Disegno schematico, spesso in **bianco e nero o scala di grigi**, con tratti volutamente "imperfetti" per suggerire che il design è ancora provvisorio.
- Struttura di una o più schermate collegate tra loro per rappresentare un **flusso di task** (task flow).
- **Interazione limitata**: i collegamenti sono statici, e l'utente può solo cliccare su zone predefinite.
- Adatti per testare **la comprensione della UI e del flusso**, ma non il comportamento dinamico.



Limiti

- Mancanza di vera interattività
- Esperienza parziale: l'utente "cerca" i punti cliccabili, ma non interagisce realmente con dati o contenuti.
- Non adatti a valutare **tempi, animazioni o reazioni dinamiche**.

Prototipi ad Alta Fedeltà

Def Gli High-Fidelity Prototypes sono prototipi molto simili al prodotto finale: presentano grafica, colori, font e layout realistici, e simulano il comportamento dei widget in modo accurato. Possono essere creati sia con strumenti di design interattivo sia con codice reale.

VANTAGGI:

- Offrono una **rappresentazione visiva e funzionale completa**.
- Permettono di testare elementi come colori, font, icone, feedback visivi e layout.
- Sono ideali per verificare **usabilità e coerenza estetica**.

SVANTAGGI:

- **Costosi e lenti da costruire**.
- Spesso l'attenzione degli utenti si concentra **sull'aspetto grafico** invece che sull'interazione.
- Possono indurre una **falsa sensazione di "prodotto finito"**, riducendo la disponibilità al cambiamento.

Tecnica del Wizard-of-Oz

Def La tecnica del **Wizard-of-Oz** (letteralmente “il Mago di Oz”) consiste nel **simulare parti non implementate** di un sistema interattivo con l’aiuto di un operatore umano, che interpreta il ruolo del “mago dietro le quinte”.

È utile per testare applicazioni **apparentemente complete**, anche quando mancano algoritmi o funzionalità. Viene spesso usato in contesti come il riconoscimento vocale o intelligenza artificiale, dove il mago simula le risposte del sistema.

Il mago può essere nascosto o visibile, ma la simulazione deve essere sempre dichiarata agli utenti al termine del test. Per implementare una tecnica di questo tipo sarà fondamentale:

1. Definire i compiti e gli scenari da testare.
2. Creare l’interfaccia utente (mock-up o semi-prototipo).
3. Fornire al mago un’interfaccia “di controllo” (back-office).
4. Stabilire regole di comportamento (quando e come rispondere).

VANTAGGI:

- Rapido ed economico rispetto a un prototipo completamente interattivo.
- Permette di testare concetti complessi e futuri.
- Aiuta i progettisti a comprendere meglio i **requisiti algoritmici**.

SVANTAGGI:

- Può creare **aspettative irrealistiche** (es. sistemi perfettamente accurati).
- Il ruolo del mago è **difficile**: deve imitare tempi e comportamenti del sistema reale.
- Richiede la presenza di **almeno due ricercatori** (facilitatore + mago).

Tecniche di Valutazione

Def. Le tecniche di valutazione hanno l'obiettivo di testare l'**usabilità** e la **funzionalità** di un sistema.

L'attività di valutazione richiede un artefatto, che può essere una simulazione, un prototipo o un'implementazione completa. La valutazione dovrebbe essere considerata in tutte le fasi del ciclo di vita del design e può avvenire in laboratorio, sul campo, e/o in collaborazione con gli utenti.

Gli obiettivi principali sono:

- Valutare l'estensione della funzionalità del sistema.
- Valutare l'effetto dell'interfaccia sull'utente.
- Identificare problemi specifici.

Una valutazione efficace ha bisogno di almeno 3 cose:

1. **Partecipanti**: dovrebbero essere rappresentativi degli utenti finali. È preferibile che abbiano un'esperienza simile tra loro (omogenei) e una conoscenza del dominio e del tipo di sistemi in esame.
2. **Scenario**: Descrive una situazione potenzialmente reale in cui i partecipanti si immedesimano. Aiuta a identificare utenti, azioni, strumenti e contesti.
3. **Task**: Sono i singoli compiti che si chiede ai partecipanti di svolgere.

Valutazione Qualitativa/Soggettiva

I metodi osservazionali fanno parte delle tecniche di valutazione che in genere portano a misure qualitative, le quali possono rivelare dettagli non determinabili da misure numeriche. I principali **Stili di Valutazione** sono:

- **Studi in Laboratorio (Lab Studies):** Consentono la disponibilità di attrezzature specialistiche ed un ambiente ininterrotto. Tuttavia, la mancanza di contesto e la difficoltà nell'osservare più utenti che cooperano possono essere punti a sfavore. Questa tecnica è ideale quando la posizione del sistema è pericolosa o impraticabile. Non deve essere svolta in un vero laboratorio, l'importante è che sia in un ambiente diverso da quello in cui di solito operano gli utenti.
- **Studi sul Campo (Field Studies):** Prevede osservazioni "in the wild", ovvero in un ambiente naturale, mantenendo un contesto reale (anche se l'osservazione potrebbe alterarlo). Tuttavia, in un ambiente non controllato, gli utenti potrebbero essere più propensi a distrazioni o rumore. Questa tecnica è ideale quando il contesto è cruciale, o per studi longitudinali.

Metodi Osservazionali Specifici

1. Think Aloud (Pensare ad Alta Voce): L'utente viene osservato mentre esegue un task e gli viene chiesto di descrivere ciò che sta facendo, perché lo sta facendo e cosa pensa che stia succedendo.

- **Vantaggi:** Semplicità (richiede poca esperienza), fornisce informazioni utili, può mostrare come viene effettivamente utilizzato il sistema.
- **Svantaggi:** È soggettivo e selettivo, l'atto di descrivere può alterare le prestazioni del task.

2. Cooperative Evaluation (Valutazione Cooperativa): Una variante del Think Aloud in cui l'utente collabora alla valutazione. Nel Think Aloud classico invece, l'esaminatore dovrebbe essere completamente esterno, senza dare alcun tipo di suggerimento. Con la Cooperative Evaluation invece, è come se ci fosse una sorta di intervista durante la valutazione, per esplorare meglio cosa sta facendo l'utente. Sia l'utente che il valutatore possono porsi domande a vicenda durante tutto il processo. Rispetto al Think Aloud risulta meno vincolato e più facile da usare, l'utente è incoraggiato a criticare il sistema ed è più facile ottenere chiarimenti.

3. Protocol Analysis (Analisi del Protocollo): Il protocollo è la registrazione della sessione di valutazione. Tale registrazione sarà poi riesaminata per rendere meglio i dati, più chiari, magari anche con l'aiuto di tool o grafici che aiutino nell'analisi. I metodi di registrazione più comuni includono carta e penna (economico, ma limitato alla velocità di scrittura), audio (buono per il Think Aloud, ma difficile da abbinare ad altri protocolli) o video (accurato e realistico, ma necessita di attrezzature speciali e potrebbe risultare invadente)

4. Post-task Walkthroughs (Rassegne Post-task): La trascrizione (o la registrazione) della sessione viene riprodotta per il partecipante, che viene invitato a commentarla. Questa tecnica è immediata, utilissima per avere i commenti freschi in mente. È utile per identificare le ragioni delle azioni e le alternative considerate. È necessario quando il *Think Aloud* non è possibile.

ATTENZIONE! Queste sono le tecniche che dovremo utilizzare nel progetto d'esame! Possiamo usarne solo 1 per tutti i task, 1 diversa per ognuno, o anche combinarne più di una tra loro.

Valutazione Quantitativa/Oggettiva

I fattori principali in una **valutazione controllata** sono:

- **Soggetti (Partecipanti):** Un campione rappresentativo e sufficiente di utenti.
- **Variabili:** Le cose da modificare e misurare. Vengono divise in due grandi categorie:
 - **Variabile Indipendente (VI):** La caratteristica che viene modificata per produrre diverse condizioni sperimentali (es. stile dell'interfaccia, numero di voci di menu). Le VI devono essere le più semplici possibili.
 - **Variabile Dipendente (VD):** La caratteristica misurata nell'esperimento (es. tempo impiegato, numero di errori).
- **Ipotesi:** Ciò che si desidera dimostrare (un'idea provvisoria il cui valore deve essere accertato). Tale ipotesi potrà essere:
 - **Ipotesi Alternativa** se si vogliamo dimostrare che una certa variazione della VI causerà una differenza nella VD (es. cambiando una parte dell'interfaccia gli utenti ci metteranno di meno (dato quantitativo) nello svolgere lo stesso identico task).
 - **Ipotesi Nulla** se vogliamo dimostrare che non ci sarà alcuna variazione (es. nessun cambiamento con la dimensione del carattere)
- **Design Sperimentale:** Il modo in cui verrà condotto l'esperimento.

Condizioni Sperimentali

Le condizioni sperimentali sono un certo numero di situazioni che differiscono solo per il valore di alcune variabili controllate. I cambiamenti nella misura comportamentale (VD) sono attribuiti alle diverse condizioni.

- **Condizione di Controllo:** La condizione di base per il confronto.
- **Condizione Sperimentale:** Prevede la modifica di solo una VI rispetto alla condizione di controllo.

Metodi Sperimentali (Design)

La scelta del metodo sperimentale è parte del design. I principali metodi che vedremo noi sono:

1. **Between Subjects (Tra Soggetti):** Ad ogni partecipante viene assegnata **una sola** delle condizioni, quindi svolge il test una sola volta. Questa tecnica non risente del "trasferimento dell'apprendimento" ma può subire "alterazioni del risultato" semplicemente per differenze individuali tra gli utenti. Un altro svantaggio è sicuramente quello di richiedere un numero di partecipanti molto maggiore rispetto al Within Subject.
2. **Within Subjects (Entro Soggetti):** Ad ogni partecipante vengono assegnate **tutte** le condizioni, quindi esegue il test due o più volte. Richiede sicuramente meno partecipanti, quindi è meno costoso, e le differenze tra gli utenti influenzano di meno la valutazione. Tuttavia risente del "trasferimento dell'apprendimento", ciò significa che l'esperienza o l'apprendimento acquisito durante lo svolgimento della prima condizione (o test) può influenzare in modo significativo la sua performance nei test successivi.

Per ridurre l'effetto del trasferimento dell'apprendimento, si può bilanciare l'ordine delle condizioni: Metà dei partecipanti esegue il primo test con la prima condizione ed il secondo con la seconda. L'altra metà, invece, esegue il primo test sotto la seconda condizione e il secondo test sotto la prima condizione.

Esempio: Stiamo testando la stessa interfaccia ma con tema chiaro o scuro. Meta partecipanti inizia con il tema chiaro e poi usa quello scuro. L'altra meta inizia col lo scuro e poi passa al chiaro.

ATTENZIONE! Il prof consiglia di studiare bene la "valutazione quantitativa" per lo scritto

Analisi Statistica dei Dati

Una volta che un esperimento è stato condotto e i dati sulle **Variabili Dipendenti (VD)** sono stati raccolti, inizia la fase di analisi.

1. Ispezione dei Dati

Il primo passo fondamentale è l'ispezione dei dati. È prassi consolidata visualizzare i dati su un grafico, poiché aiuta a comprendere la distribuzione e, soprattutto, a ricercare eventuali **outliers**. Un outlier (o valore anomalo) è un dato che si discosta significativamente dalle altre osservazioni. Ad esempio, un partecipante che impiega un tempo estremamente lungo o commette un numero sproporzionato di errori rispetto al gruppo. Questi dati possono distorcere l'analisi complessiva. È importante identificare gli outliers per decidere se correggerli o rimuoverli, pur mantenendo una copia dei dati originali per completezza e replicabilità. Per scegliere il metodo statistico appropriato, dobbiamo considerare la natura delle variabili:

- **Variabili Indipendenti (VI):** Sono ciò che manipoliamo (es. tipo di interfaccia, dimensione del font). Sono tipicamente discrete (categorie distinte).
- **Variabili Dipendenti (VD):** Sono le misure che raccogliamo. Possono essere discrete (valori interi come il numero di errori) o continue (valori con infinite possibilità decimali all'interno di un intervallo, come il tempo impiegato). Nell'interazione Uomo Macchina molte VD sono continue e positive (> 0).

2. La Scelta del Test Statistico

L'analisi statistica, in particolare quella parametrica, si basa sulla ricerca della **distribuzione normale della variabile dipendente**. La distribuzione normale, rappresentata da una curva a campana, è un modello teorico che presuppone che la maggior parte dei valori si addensino attorno alla media con deviazioni simmetriche. Questa proprietà è cruciale per la validità di molti test statistici.

La forma della distribuzione dei dati determina quale tipo di test statistico possiamo utilizzare:

- **Test Parametrici:** Sono utilizzati se i dati soddisfano la **distribuzione normale**. Sono considerati tecniche di analisi statistica standard molto robuste. Essi offrono la massima potenza per rilevare un effetto se le loro assunzioni di distribuzione sono rispettate.
- **Test Non Parametrici:** Sono utilizzati quando i dati non seguono una distribuzione normale o quando si usano scale di misura non quantitative. Questi test non fanno assunzioni sulla forma della distribuzione dei dati e possono avvalersi anche di strumenti come la tabella di contingenza.

3. Verifica d'Ipotesi

Il culmine dell'analisi statistica è la **verifica d'ipotesi**. Lo scopo è determinare se l'effetto osservato (la differenza nelle VD tra le diverse condizioni) è reale o se è dovuto semplicemente al caso. Questo si traduce in una decisione sul rifiuto o meno dell'ipotesi nulla.

Se il test statistico mostra che la probabilità di ottenere il risultato osservato per puro caso è molto bassa (ad esempio, inferiore all'1%), allora possiamo rifiutare l'ipotesi nulla. Si dichiara quindi che "possiamo, al 99%, rifiutare l'ipotesi nulla", concludendo che l'effetto della Variabile Indipendente è statisticamente significativo. Rifiutare l'ipotesi nulla significa, in sostanza, accettare l'ipotesi alternativa: che la manipolazione introdotta (ad esempio, il nuovo design) ha effettivamente causato il cambiamento nella performance dell'utente.

Tecniche di Valutazione Expert Based

Nel ciclo di sviluppo del software, è fondamentale valutare frequentemente. Un principio chiave è che più tardi si individua un errore nel design o nell'interazione, più costoso diventa correggerlo. Tuttavia, sorge un problema pratico: sebbene il feedback degli utenti finali sia prezioso, coinvolgerli in *tutte* le fasi di valutazione è estremamente costoso e dispendioso in termini di tempo.

Per ovviare a questo, si ricorre alle valutazioni "expert-based", ovvero valutazioni svolte da **soggetti interni al gruppo di lavoro**, senza coinvolgere gli utenti finali. L'obiettivo è individuare aspetti dell'interfaccia che possono causare difficoltà, ad esempio perché violano noti principi cognitivi o ignorano risultati empirici già accettati e consolidati nel campo. Questo approccio presenta chiari benefici e limiti:

- **PRO:** Sono metodi economici e possono essere utilizzati in qualsiasi fase del progetto e dello sviluppo.
- **CONTRO:** Non valutano il vero uso del sistema da parte di utenti reali in contesti reali. Valutano piuttosto l'aderenza teorica del design ai principi e alle euristiche conosciute.

Esempio: gli utenti interni ai gruppi di lavoro non possono testare l'interfaccia con tecniche qualitative semplici, come il Think Aloud, poiché hanno sviluppato loro l'interfaccia... dunque sapranno già tutto quello che devono fare per svolgere i task!

1. Cognitive Walkthrough (CW)

Def Il Cognitive Walkthrough è una tecnica che **valuta il design** analizzando quanto bene supporta l'utente nell'apprendimento di un compito (focus sulla *learnability*).

In altre parole, valuta **quanto è intuitivo svolgere un determinato task**. Di solito, viene eseguito da un esperto di psicologia il quale, metaforicamente, "passeggia" (walks through) attraverso il design, schermata dopo schermata, utilizzando principi psicologici per identificare potenziali problemi di apprendimento.

Per eseguire un Cognitive Walkthrough efficace, l'esperto ha bisogno di:

1. **Specifiche del sistema:** Come è fatta e come funziona l'interfaccia.
2. **Descrizione dell'utente tipo:** Il profilo dell'utente target (es. "docente universitario").
3. **Descrizione di un task:** Un compito specifico che l'utente deve completare
4. **Sequenza dei passi:** L'elenco preciso delle azioni necessarie per completare il task.

Per ogni singolo passo della sequenza, l'esperto deve cercare di rispondere a 4 domande fondamentali:

1. **L'effetto dell'azione che l'utente compie è lo stesso dell'obiettivo che l'utente ha in mente in quel momento?**
2. **L'utente può vedere che l'azione richiesta è disponibile?**
3. **L'utente capirà che quella specifica azione è quella giusta da eseguire per il suo obiettivo?**
4. **Dopo aver eseguito l'azione, l'utente capirà il feedback ricevuto e saprà di essere sulla strada giusta?**

Tutti i problemi e le "storie di fallimento" a queste domande vengono **riportati su un form**, indicandone anche il grado di severità.

Cognitive Walkthrough - Esempio

Vediamo un'applicazione pratica del CW sul compito: "Un host desidera abilitare la condivisione dello schermo per tutti i partecipanti". I passaggi identificati sono:

1. Cliccare sull'icona a forma di freccia in su, sopra al pulsante "Share Screen".
2. Accedere alle "Advanced Sharing Options".
3. Selezionare l'opzione "All Participants".

Analisi del Walkthrough:

- **Step 1 (Cliccare la freccia in su):**
 - *Domanda 1 (Obiettivo):* Parzialmente. L'utente vuole cambiare le *impostazioni*, ma la freccia è sul pulsante "Share Screen", che sembra riferirsi solo all'azione di *condividere* lo schermo dell'host.
 - *Domanda 2 (Visibilità):* Sì e no. Il pulsante "Share Screen" è visibile, ma la piccola icona a forma di freccia in su non è chiaramente visibile.
 - *Domanda 3 (Comprensione):* Probabilmente no. L'utente potrebbe non capire che è *quella* freccina a gestire le impostazioni e potrebbe cercare altrove (es. nel menu "Security" o nelle impostazioni generali).
 - *Domanda 4 (Feedback):* Sì e no. Cliccando appare un menu, ma l'opzione "avanzata" è piccola e non immediatamente visibile.
 - *Feedback Step 1:* Questo passaggio è confuso; l'accesso alle impostazioni non è chiaro.
- **Step 2 (Accedere a "Advanced Sharing Options"):**
 - *Domanda 1 (Obiettivo):* Sì, in teoria. L'opzione avanzata dovrebbe contenere il controllo, ma è nascosta.
 - *Domanda 2 (Visibilità):* No, non immediatamente. L'opzione è scritta in piccolo in fondo al menu.
 - *Domanda 3 (Comprensione):* Probabilmente no. Il termine "Advanced" (Avanzate) potrebbe spaventare o non suggerire chiaramente che contiene il controllo dei permessi. Potrebbe essere interpretato come relativo a impostazioni tecniche di qualità video.
 - *Feedback Step 2:* Questo passaggio può essere facilmente trascurato.
- **Step 3 (Selezionare "All Participants"):**
 - *Domanda 1 (Obiettivo):* Sì. L'effetto (abilitare tutti) corrisponde all'obiettivo.
 - *Domanda 2 (Visibilità):* Sì. Una volta nel pannello, l'opzione è visibile.
 - *Domanda 3 (Comprensione):* Sì. La scelta tra "Only Host" e "All Participants" è molto chiara.
 - *Domanda 4 (Feedback):* Sì. La selezione è un feedback chiaro e si applica immediatamente.
 - *Feedback Step 3:* Il problema qui non è l'interfaccia, ma l'impostazione predefinita ("Only Host") che non viene comunicata attivamente all'utente.

Problemi Identificati dal CW (Sintesi):

1. **Problema Step 1:** L'accesso alle impostazioni tramite la piccola freccia sul pulsante "Share Screen" non è chiaro. (Soluzione: un segnale visivo più evidente).
2. **Problema Step 2:** Le "Advanced Sharing Options" sono nascoste e scritte in piccolo. (Soluzione: spostarle in posizione più prominente o aggiungere un tooltip).
3. **Problema Step 3:** L'impostazione predefinita è "Only Host" e non c'è nessuna notifica che lo segnali all'host. (Soluzione: aggiungere un avviso o un suggerimento).

2. Heuristic Evaluation (HE)

Def Jakob Nielsen e Rolf Molich hanno proposto le **10 euristiche di usabilità più utilizzate**. Nel processo di valutazione, il valutatore esamina il design e verifica se viola una o più di queste euristiche, riportando per ogni violazione la severità del problema.

Un'euristica è una linea guida, un principio generale o una regola pratica semplice ed efficiente, usata per guidare le decisioni di progetto o per criticare quelle già prese.

Le 10 Euristiche di Nielsen

1. **Visibilità dello stato del sistema:** L'utente deve SEMPRE sapere cosa sta succedendo.
Esempio: su una mappa la freccia "Tu sei qui"
2. **Corrispondenza tra il sistema e il mondo reale:** Il sistema deve usare il linguaggio dell'utente, non si può utilizzare un linguaggio tecnico.
3. **Libertà e controllo da parte degli utenti:** Gli utenti spesso effettuano operazioni per sbaglio... Deve sempre poter annullare in un singolo passaggio l'operazione in corso.
Esempio: premendo sul logo del sito/app mi riporta direttamente all'home page, senza dover premere undo 50 volte)
4. **Coerenza e standard:** Convenzioni identiche per azioni identiche.
5. **Prevenzione degli errori:** Meglio prevenire un errore che fornire un buon messaggio di errore.
Esempio: per inserire le date ormai si utilizzano delle interfacce che replicano dei calendari. Sarà quindi impossibile selezionare il 30 febbraio, o il giorno sbagliato della settimana)
6. **Riconoscere è meglio che Ricordare:** Le opzioni devono essere visibili, non da ricordare a memoria.
7. **Flessibilità ed efficienza d'uso:** Scorciatoie per utenti esperti, ma facilità per i neofiti.
Esempio: possibilità di fissare le chat, oppure mettere delle scorciatoie sulla schermata home, acquisti periodici su amazon, articoli preferiti, etc...
8. **Design minimalista ed estetico:** L'interfaccia non deve contenere informazioni irrilevanti.
Esempio: Un unico menu gigantesco, tutte le voci mischiate e non organizzate in modo chiaro. Se invece dividiamo le opzioni in categorie/sottomenu, in ogni schermata vediamo solo quello di cui abbiamo bisogno.
9. **Aiutare gli utenti a Riconoscere, Diagnosticare e Correggere gli Errori:** Messaggi di errore chiari.
10. **Accesso facile alla Documentazione.**

Heuristic Evaluation - Esempio

Applichiamo le euristiche allo stesso problema di Zoom di prima.

- **H1: Visibilità dello stato del sistema**
 - *Problema:* L'icona "Share Screen" non mostra chiaramente che *contiene* le impostazioni avanzate. Lo stato (attuale permesso di condivisione) non è visibile a colpo d'occhio.
- **H2: Corrispondenza mondo reale**
 - *Problema:* L'etichetta "Advanced Sharing Options" è vaga. Il termine "Advanced" può risultare tecnico e non intuitivo per un utente comune. (Soluzione: Rinominare, es. "Gestione Condivisione").
- **H3: Controllo e libertà utente**
 - *Problema:* Il sistema non avvisa che l'impostazione predefinita è "Only Host". L'utente non ha un controllo *intuitivo* perché l'impostazione di default non è comunicata esplicitamente.
- **H4: Coerenza e standard**
 - *Problema:* L'icona "Share Screen" è associata all'azione di *condividere*, non alla *gestione* delle autorizzazioni. Usare la stessa icona per due funzioni diverse (azione vs impostazioni) genera confusione e viola la coerenza.
- **H5: Prevenzione degli errori**
 - *Problema:* La mancanza di notifica sull'impostazione "Only Host" porta quasi certamente a un errore (lentezza o confusione durante la call quando qualcuno prova a condividere). (Soluzione: un promemoria all'inizio della call).
- **H6: Riconoscere piuttosto che ricordare**
 - *Problema:* L'host deve *ricordare* (recall) di controllare queste impostazioni ogni volta. (Soluzione: aggiungere un'icona che indichi l'impostazione corrente *direttamente* sulla barra, favorendo il *riconoscimento* (recognition)).
- **H7: Flessibilità ed efficienza d'uso**
 - *Problema:* Non c'è un modo rapido (scorciatoia) per cambiare l'autorizzazione senza passare per 3 click nelle opzioni avanzate. Questo interrompe il flusso della call.
- **H8: Estetica e design minimalista**
 - *Problema:* L'opzione "Advanced" è nascosta e scritta in piccolo, aggiungendo confusione (clutter) invece che chiarezza.
- **H9: Aiutare a correggere gli errori**
 - *Problema:* Se un *partecipante* prova a condividere (e non può), non riceve un feedback chiaro sul *perché* (es. "L'host non ha abilitato la condivisione"). (Soluzione: un messaggio chiaro per l'utente che tenta l'azione).
- **H10: Aiuto e documentazione**
 - *Problema:* Zoom non offre un tooltip o un link diretto che spieghi cosa facciano le "Advanced Sharing Options".

Model-based Evaluation

Si prendono modelli cognitivi o computazionali già esistenti e si testano per valutare o prevedere l'interazione. Anche questa tecnica si occupa di testare l'interfaccia. Ad esempio potrà confrontare il tempo che l'esperto ha impiegato per svolgere il task con quello previsto dal modello scelto.

I principali modelli che vediamo sono:

- **GOMS (Goals, Operators, Methods, Selections):** Un modello per prevedere le performance dell'utente (es. tempi di esecuzione) nell'uso di un'interfaccia.
- **KLM (Keystroke-Level Model):** Un modello più semplice che scompone i compiti in azioni fisiche di basso livello (uso di tastiera, mouse) per prevederne i tempi di esecuzione.
- **Dialog Models:** Usati per analizzare formalmente le sequenze di dialogo (interazioni) per trovare stati irraggiungibili o dialoghi circolari (loop) .
- **Legge di Fitts:** È un modello che descrive la relazione tra la velocità e la precisione del movimento umano. Calcola il tempo necessario per muovere rapidamente un puntatore (es. il mouse o il dito su un touchscreen) da una posizione iniziale a un'area target finale. Nella progettazione di interfacce, la legge di Fitts stabilisce che i target più grandi e più vicini sono i più veloci da selezionare. È quindi un'euristica quantificabile per posizionare pulsanti importanti e interattivi nelle aree più accessibili (ad esempio, ai bordi dello schermo o vicino al puntatore corrente) e con dimensioni adeguate.

Review-based Evaluation

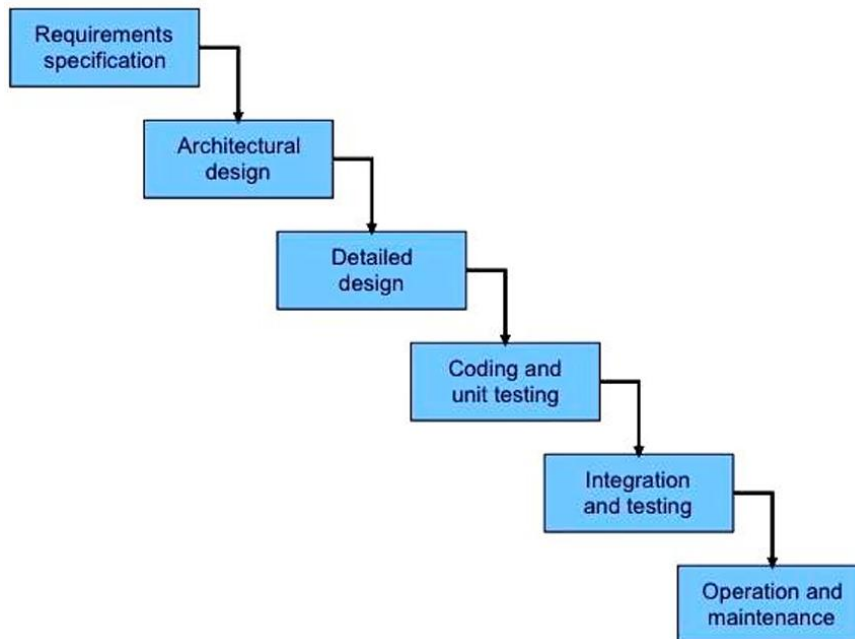
In questo approccio, si utilizzano i **risultati di studi precedenti** (dalla letteratura scientifica) per supportare o rifiutare parti specifiche di un design. È fondamentale che un esperto si assicuri che i risultati di quegli studi siano **trasferibili** al nuovo contesto e che le assunzioni fatte siano corrette.

ATTENZIONE! Per il Progetto è necessario utilizzare CW o HE. Le altre tecniche non sono applicabile al nostro lavoro.

Progetto e Sviluppo di Sistemi Interattivi

Questa lezione si concentra sui modelli di ciclo di vita del software, in particolare analizzando i limiti del modello tradizionale Waterfall e introducendo l'approccio Agile più adatto allo sviluppo di sistemi interattivi.

Waterfall Model



Il modello Waterfall (o "a cascata"), noto per la sua struttura rigida e sequenziale, prevede una serie di fasi che procedono linearmente:

1. **Specifiche dei Requisiti:** Il progettista e il cliente cercano di definire ciò che il sistema deve fornire. I requisiti possono essere espressi in linguaggio naturale o in linguaggi più precisi, come quelli forniti dalla task analysis.
2. **Progettazione Architettuale:** Si elabora una descrizione di alto livello su come il sistema fornirà i servizi richiesti. Il sistema viene suddiviso in componenti principali e si definiscono le loro interrelazioni, soddisfacendo sia i requisiti funzionali che quelli non funzionali.
3. **Progettazione Dettagliata:** È un affinamento dei componenti architettureali per identificare i singoli moduli che dovranno essere implementati separatamente. Questo affinamento è guidato in particolare dai requisiti non funzionali.
4. **Coding and Unit Testing** delle singole componenti
5. **Integrazione e Test:** Si mettono insieme tutte le componenti per formare il servizio (quasi) finale e poi si testa.
6. **Operatività e Manutenzione.**

Limiti: Il modello Waterfall presuppone una sequenza lineare di attività. Tuttavia, per i sistemi interattivi, questa ipotesi spesso fallisce perché lo sviluppo di una fase può fornire informazioni utili anche per le fasi precedenti. Spesso quindi capita di dover tornare indietro e svolgere molti cicli di feedback tra le fasi.

Inoltre, poiché lo sviluppo di software interattivo è principalmente un processo di **design creativo**, la separazione rigida del design dallo sviluppo (tipica del Waterfall) fallisce. Le specifiche di design cambiano continuamente per una serie di motivi:

- Cambiamento del contesto di utilizzo. (es. i computer hardware cambiano di continuo, si evolvono)
- Una migliore comprensione del problema che emerge durante lo sviluppo.
- La scoperta di utilizzi inattesi (o *unexpected usage*).
- Nuovi requisiti di business.
- L'avanzamento costante della base tecnologica.
- L'emergere di nuove idee

Iterative Design e Prototyping

La soluzione ai problemi intrinseci del Metodo Waterfall (requisiti incompleti e cambiamenti costanti) è l'approccio Iterativo. Il design iterativo riconosce che il processo è ciclico e che la conoscenza del sistema migliora ad ogni ciclo. Un elemento chiave di questo approccio è l'uso dei Prototypes.

I Prototipi sono versioni incomplete del sistema che simulano solo alcune funzionalità, senza implementarle veramente. Questo ci permette di testare come funzionerà il prodotto finale senza doverlo implementare realmente. Una volta capito se il prototipo funzionerà o meno si potrà passare allo sviluppo effettivo.

(Come abbiamo già visto qualche lezione fa) i principali tipi di Prototipi sono:

- **Throw-away (usa e getta):** Creati per esplorare un concetto e poi scartati.
- **Incremental (incrementali):** Il sistema viene costruito per piccole parti, e ogni parte è un prototipo che viene integrato.
- **Evolutionary (evolutivi):** Il prototipo iniziale evolve gradualmente fino a diventare il sistema finale.

Metodologia Agile

Agile è un processo iterativo di gestione del progetto nato per superare i limiti dei modelli predittivi (come Waterfall) nell'era del software in rapida evoluzione. Agile incoraggia:

- L'ispezione frequente e l'adattamento.
- Il lavoro di squadra e l'auto-organizzazione.
- La consegna rapida di software di alta qualità.
- L'allineamento costante dello sviluppo con le esigenze del cliente.

Le metodologie Agile sono intrinsecamente diverse dal Waterfall per due aspetti fondamentali:

1. **Adattive piuttosto che Predittive:** I metodi Agile sono adattivi. Accolgono volentieri il cambiamento, anche fino al punto di cambiare se stessi (auto-adattamento del processo).
2. **Orientate alle Persone piuttosto che al Processo:** I metodi Agile sono orientati alle persone. Il ruolo del processo è quello di supportare attivamente il team di sviluppo nel suo lavoro

Metodo Agile UCD

L'Agile UCD è un approccio ibrido tra le metodologia Agile e UCD (User-Centered design), molto più efficace nello sviluppo di sistemi interattivi. La necessità è quella di **fondere l'orientamento al cambiamento** di Agile **con il focus sull'utente** dell'UCD. L'integrazione di UCD in Agile si basa su principi gestionali specifici:

- **Design e Sviluppo Concorrente:** Le attività di design e sviluppo devono avvenire in parallelo.
- **Gestione del Tempo (Slot Fissi):** Non si utilizzano stime di tempo dettagliate per l'intero progetto. I lassi di tempo (*time slots*) per le iterazioni sono **fissi**; sono gli **obiettivi** a essere adattati
- **Flessibilità al Cambiamento:** È sempre possibile cambiare gli obiettivi o le specifiche del prodotto.
- **Rilascio Frequente:** Il rilascio del prodotto o delle sue funzionalità deve avvenire **presto e spesso**
- **Competenze Specialistiche:** Si valorizzano la specializzazione e le competenze specifiche (*skills*) all'interno del team.

Cicli di Sviluppo

Il processo AgileUCD è suddiviso in cicli (o **sprint**), la cui durata tipica è di **2-4 settimane** dall'inizio al completamento, con date di fine fisse.

Ciclo Zero

È la fase iniziale di **ricerca, analisi e strategia di design di alto livello**. Questa fase è pre-esistente nel tradizionale UCD e getta le basi per tutte le iterazioni future. È proprio la prima iterazione, NON si può basare su design precedenti. Le attività includono:

- Definizione degli **obiettivi di prodotto**.
- Creazione di una **visione condivisa** del prodotto.
- Conduzione di **ricerca contestuale**.
- Creazione di **Personas** e **Scenari** che rappresentano l'utente tipo e i suoi compiti).

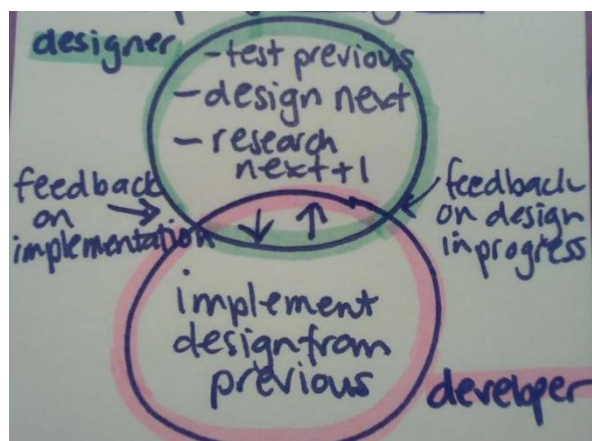
ATTENZIONE! Il Ciclo Zero nell'AgileUCD **non produce software funzionante** o una "prima versione del sistema" da parte dei developer. Produce invece gli artefatti di design di alto livello che serviranno come base per la prima iterazione di codice.

Ciclo Intermedio

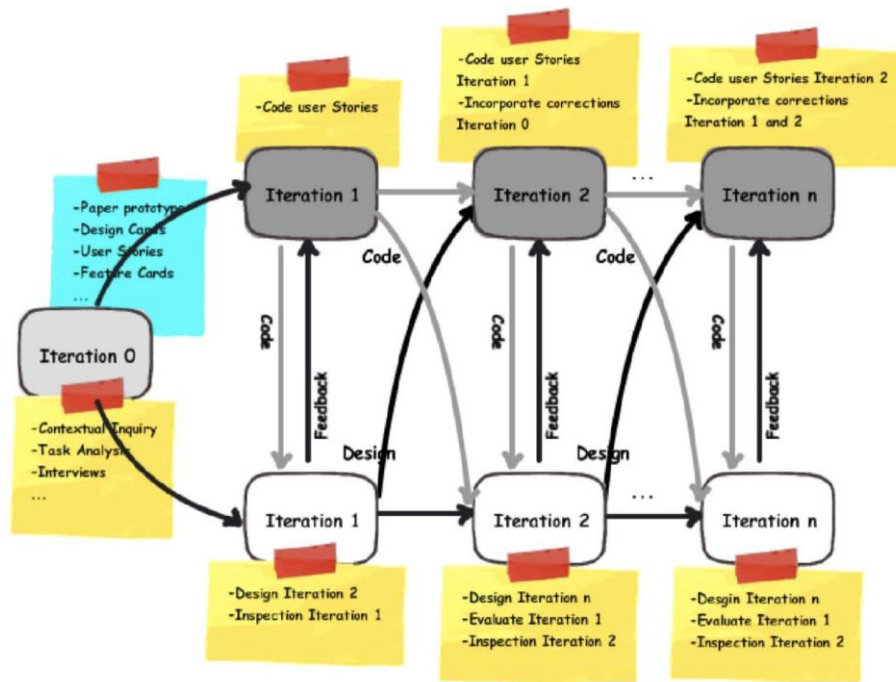
Anche detto "ciclo 1". Qualsiasi ciclo eseguito dopo lo zero avrà questa struttura:

- **Designer:** Testano con gli utenti il sistema che è stato realizzato al ciclo precedente dagli sviluppatori. Dai risultati dei test fanno il design che gli sviluppatori dovranno implementare nel prossimo ciclo.
- **Developer:** Implementano il design progettato dai designer nel ciclo precedente.

Feedback: C'è un flusso CONTINUO di feedback tra i team di developers e progettisti (designers). DURANTE ogni fase si scambiano le opinioni, per migliorare nel ciclo successivo.



Esempio:

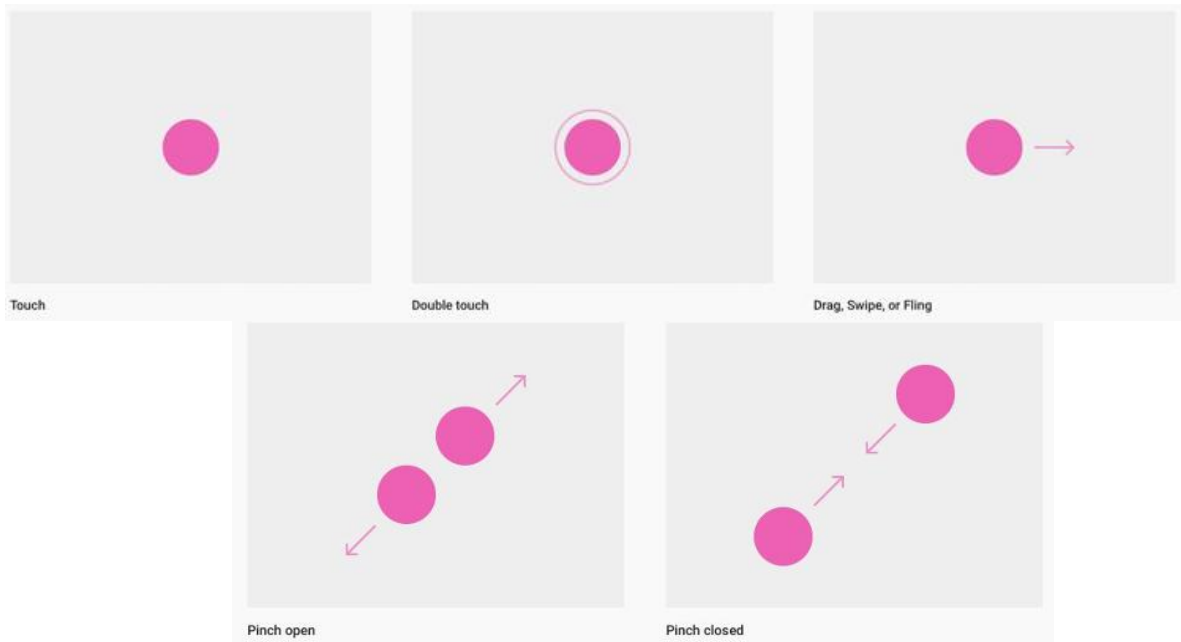


Best Practices per l'Agile UCD

Per ottimizzare l'integrazione tra design e sviluppo in Agile, sono raccomandate diverse pratiche:

- **Risorse e Collaborazione:** Prevedere più risorse di design (more design resources). Promuovere la collaborazione tra le discipline (collaboration between disciplines), abbattendo i silos tra designer e sviluppatori.
- **Organizzazione del Lavoro:** Scomporre il design in parti che si adattino alle dimensioni del ciclo (breaking design into cycle sizes). Utilizzare Feature Cards (schede delle funzionalità). Riunioni quotidiane in piedi (daily standup meetings) pubblicamente mostrate.
- **Documentazione e Feedback:** Dare maggiore enfasi alla documentazione attraverso il testing. Ridurre la dipendenza dal feedback da "ambasciatori utente". Eseguire un rilascio frequente (frequent release).

Interfacce Android



I principi fondamentali per la progettazione di interfacce utente Android sono volti a rendere l'applicazione intuitiva, reattiva e coerente. Si cerca di **sfruttare il più possibile la Conoscenza degli Utenti** (Leverage on Users' Knowledge). L'interfaccia deve basarsi su elementi e interazioni che gli utenti già conoscono:

- **Widget Standard:** Utilizzare componenti standard (es. pulsanti, campi di testo, progress bar per attese più lunghe di qualche secondo, etc...)
- **Gesture Standard:** Adottare gesti comuni per azioni standard (es. scorrere per scartare, pizzicare per zoomare, etc...).
 - **Touch:** Pressione di un dito per selezionare.
 - **Doppio Tocco (Double touch):** Doppia pressione di un dito per fare Zoom in.
 - **Trascinamento/Scorrimento (Drag, Swipe, or Fling):** Pressione, movimento, sollevamento di un dito per scorrere.
 - **Pinch Open:** Pressione di due dita, movimento verso l'esterno, sollevamento per Zoom in.
 - **Pinch Closed:** Pressione di due dita, movimento verso l'interno, sollevamento per Zoom out.
- **Metafore:** Utilizzare metafore visive (es. un interruttore per lo stato On/Off)

Gli elementi e le azioni importanti devono essere facili da trovare e l'interfaccia deve sempre reagire all'input dell'utente.

- **Non Nascondere i Controlli:** I controlli importanti non dovrebbero MAI essere nascosti o richiedere azioni complesse per essere rivelati.
- **Manipolazione Diretta e Reazione:** L'utente deve percepire che sta manipolando direttamente gli oggetti sullo schermo e ricevere un feedback immediato e pertinente per ogni azione.
- **Barre di Progresso (Progress Bars):** Fornire un feedback visivo sui processi che richiedono più di qualche secondo. Tipi comuni includono Determinate, Indeterminate, Buffer e Query Indeterminate and Determinat.

Users in Control

L'utente deve sentirsi padrone dell'applicazione e delle sue azioni. Deve sempre sapere in quale schermata si trova (es. dopo che lo schermo va in standby e lo riaccendo devo capire dove mi trovo)

- **Navigazione Consistente:**
 - **Action Bar/Top App Bar:** Fornisce navigazione consistente, visibilità dell'identità dell'app e rende le azioni principali prominenti e accessibili (es. Cerca, Menu).
 - **Tasto Indietro (Back):** Permette la navigazione inversa cronologica tra le schermate.
 - **Navigazione Verso l'Alto:** Permette di muoversi all'interno della gerarchia dell'app (es. da un dettaglio a una lista).
- **Annulla (Undo):** Consentire all'utente di annullare facilmente le azioni potenzialmente distruttive o involontarie.
- **Opzioni di Annullamento:** Fornire sempre un'opzione chiara per annullare operazioni o interrompere.

Coerenza

L'esperienza utente deve essere coerente nell'uso di gesti, widget e temi.

- **Stesso Gesto, Stessa Azione:** Un gesto dovrebbe sempre eseguire la stessa azione in tutta l'applicazione. Altrimenti rischiamo incoerenza e difficoltà nell'uso della UI.
- **Stessa Funzione, Stesso Widget:** Lo stesso tipo di funzione (es. menu a tendina) dovrebbe essere implementato con lo stesso widget.
- **Evitare Modalità (Modes):** Le interfacce modali (dove un elemento cambia funzione a seconda di uno stato) confondono l'utente. L'interfaccia dovrebbe mantenere le funzioni inalterate.
- **Temi:** Garantire che il tema (es. Light/Dark) sia coerente in tutti gli elementi dell'interfaccia.
- **Animazioni giustificate da un'Azion:** Ogni cosa è statica. Non si muove. Se qualcosa si muove è per rappresentare un'azione, qualcosa che si sta compiendo in questo istante (es. progress bars)

Personalizzazione e Memoria

L'interfaccia dovrebbe adattarsi all'utente e ai contesti, minimizzando lo sforzo di memoria.

- **Memorizzazione vs. Selezione:** Ridurre il carico cognitivo preferendo la selezione (mostrando le opzioni) alla memorizzazione (richiedendo all'utente di ricordare le informazioni).
- **Apprendimento e Suggerimento:** L'app può imparare le preferenze dell'utente e offrire suggerimenti pertinenti (es. suggerimenti di completamento automatico).
- **Defaults:** Impostare valori predefiniti intelligenti e contestuali.
- **Personalizzazione:** Consentire all'utente di personalizzare l'esperienza.

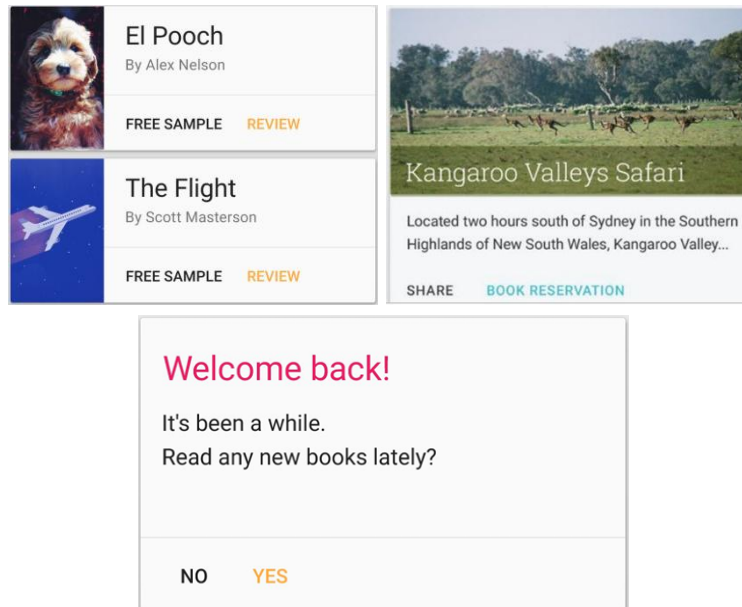
Material Design Components (Widgets)

Material Design 3 definisce una serie di componenti (widget) che guidano la progettazione dell'interfaccia.

Cards

Le Cards sono contenitori utilizzati per visualizzare contenuti e azioni relativi a un singolo argomento.

Possono contenere una varietà di elementi, inclusi headline, testo, divisori, media, icone, immagini e pulsanti. Possono essere disposte insieme in una griglia, un elenco verticale o un carosello. Una Card è solitamente un unico grande target che può essere toccato per mostrare una schermata di dettaglio più ampia.



Lists

Le Lists sono indici verticali e continui di testo e immagini.

Il loro scopo è aiutare gli utenti a trovare un elemento specifico e ad agire su di esso. Sono disponibili in tre dimensioni: una, due o tre linee. È fondamentale mantenere gli elementi brevi e facili da scansionare e mostrare: icone, testo e azioni in un formato coerente, allineando gli elementi visivi al bordo principale. L'ordine degli elementi dovrebbe seguire una logica (come alfabetico o numerico).

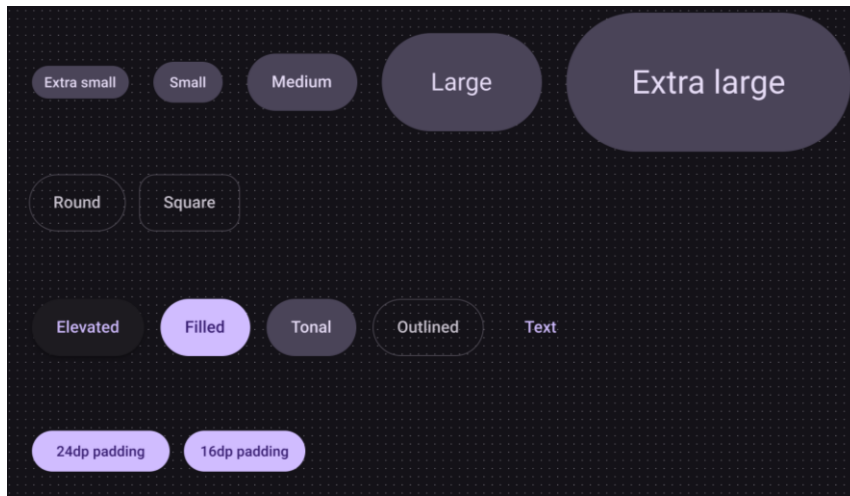
Ali Connors Brunch this weekend? I'll be in your neighborhood doing errands ...	15m ☆
me, Scott, Jennifer Summer BBQ Aw dang. Wish I could but I'm outta town ...	2h ☆
Sandra Adams Oui Oui Do you have Paris recommendations? Hav...	6h ☆
Trevor Hansen Order Confirmation Thank you for your recent order from Stev...	12h ☆

ATTENZIONE! È importantissimo NON tipizzare le liste. Non ci deve essere una colonna Nome, Cognome, Ora... La list deve essere semplice abbastanza da permettere all'utente di capire da solo cos'è ogni campo

Buttons

I Buttons (pulsanti) comunicano le azioni che gli utenti possono intraprendere.

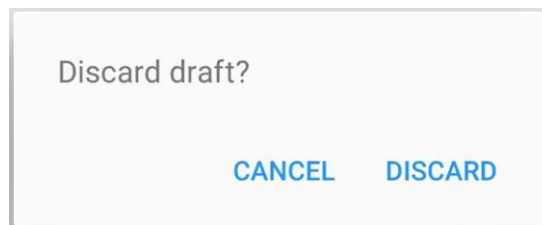
I contenitori dei pulsanti hanno angoli completamente arrotondati e sono larghi abbastanza da adattarsi al testo dell'etichetta. Sono usati in diversi contesti, come dialoghi, finestre modali, form, card e barre degli strumenti. Se presenti, le icone DEVONO essere posizionate PRIMA del testo.



Alerts

Gli Alerts (Avvisi) comunicano all'utente situazioni critiche o una conferma di decisione irreparabile che l'utente deve prendere.

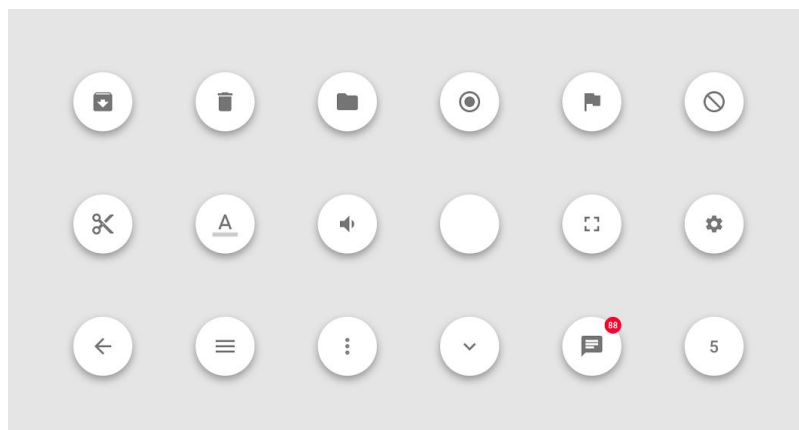
Sono tipicamente generati dal sistema e fortemente sconsigliati. Bloccano completamente l'applicazione fino a quando non si seleziona una delle opzioni proposte dall'Alert stesso. Vanno utilizzati SOLO quando si sta svolgendo un task con conseguenze gravi, distruttive (es. pagamento o l'eliminazione definitiva di un file)



Floating Action Buttons (FAB)

I FAB promuovono l'azione più comune o importante in una schermata, come creare, aggiungere ai preferiti, condividere o avviare un processo.

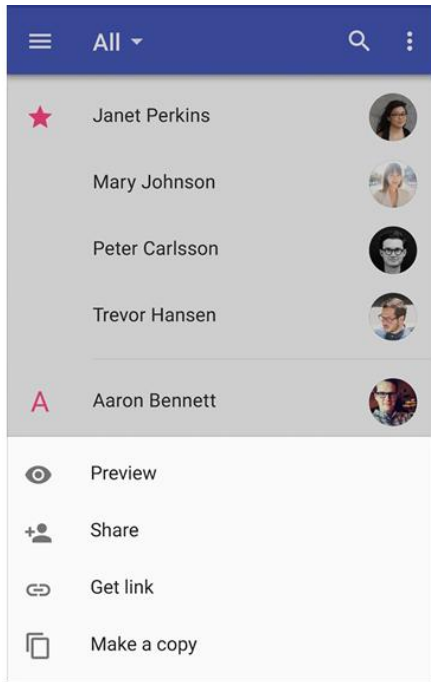
Hanno un'icona chiara e comprensibile. Persistono anche quando il contenuto scorre e sono posizionati sopra la navigation bar o annidati al suo interno. Vengono visualizzati in un contenitore quadrato arrotondato.



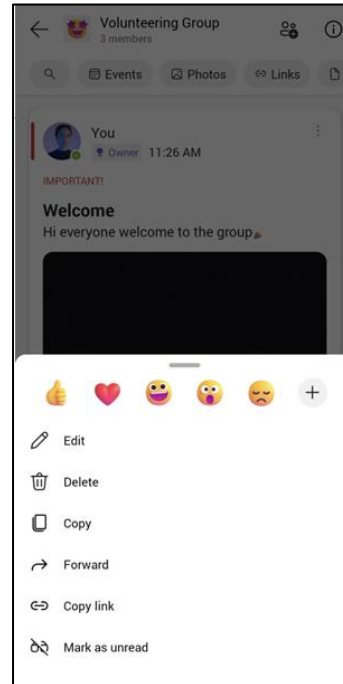
Bottom Sheets

I **Bottom Sheets** visualizzano contenuti e azioni supplementari. Un bottom sheet dovrebbe apparire solo se sollecitato dall'utente. Esistono due tipi:

- **Standard**: visualizza contenuto che è di complemento al contenuto primario della schermata. Quando appare rimane vincolato al suolo. Ha quelle dimensioni lì e non puoi “alzarsi”
- **Modal**: è un'alternativa a menu inline o dialoghi semplici su mobile e può essere tirata su per coprire l'intero schermo (tall) e scorre internamente.



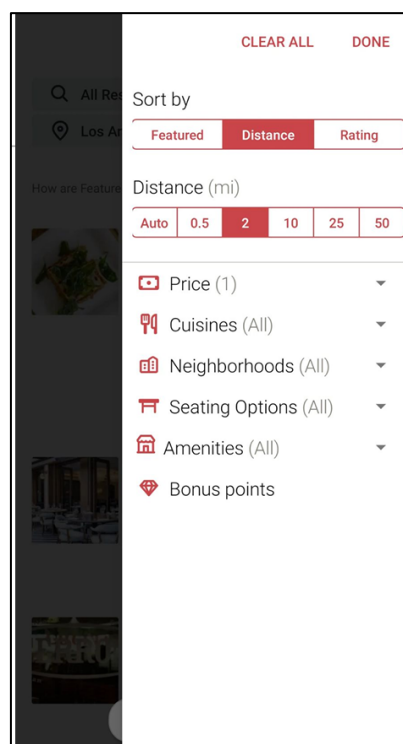
Standard



Modale

Side Sheets

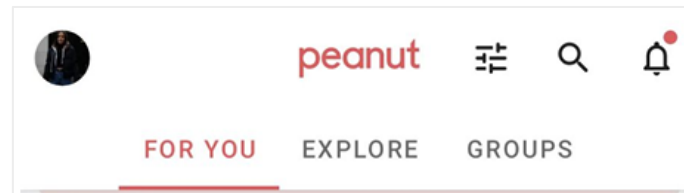
I **Side Sheets** mostrano contenuto secondario ancorato al lato dello schermo. Sono come i Bottom Sheets ma appaiono di lato anziché dal basso. (es. burger menu)



Tabs

Le Tabs organizzano il contenuto raggruppandolo in categorie utili, disposte su diverse schermate e viste.

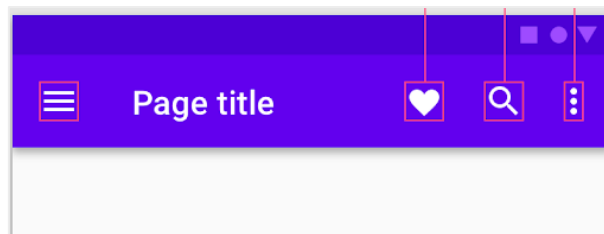
Sono collocate in alto, sotto la top app bar. Possono avere un'etichetta e un'icona (opzionale). La tab attiva è indicata con una sottolineatura e un cambio di colore per il testo e l'icona. Gli utenti possono navigare tra le tab anche scorrendo a sinistra o a destra.



Top App Bars

Le Top App Bars contengono il titolo della schermata e le azioni correlate, fornendo anche navigazione, headline della schermata e azioni.

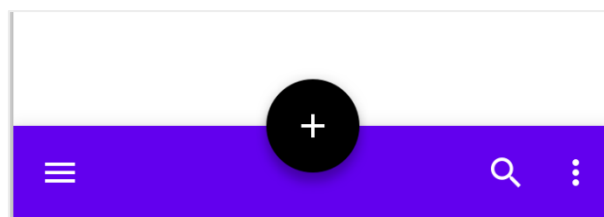
Possono contenere un'icona di navigazione principale opzionale in due forme: un'icona di menu (hamburger) o una freccia indietro. Lo scorrimento verso il basso nasconde la barra dell'app, mentre lo scorrimento verso l'alto la rivela.



Bottom App Bars

Le Bottom App Bars visualizzano navigazione e azioni chiave.

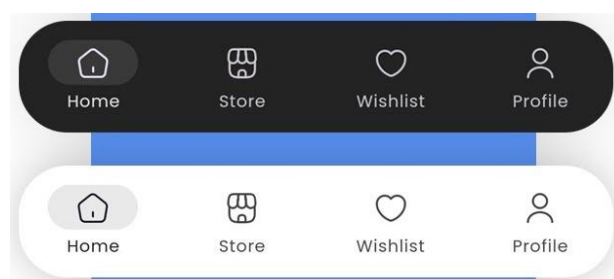
Possono contenere fino a quattro pulsanti icona e un FAB. Non sono barre di navigazione e non devono essere usate per app che già usano una navigation bar o su schermate con una o nessuna azione. Se utilizzate per un'azione distruttiva, dovrebbero attivare una snackbar per la conferma o l'annullamento.



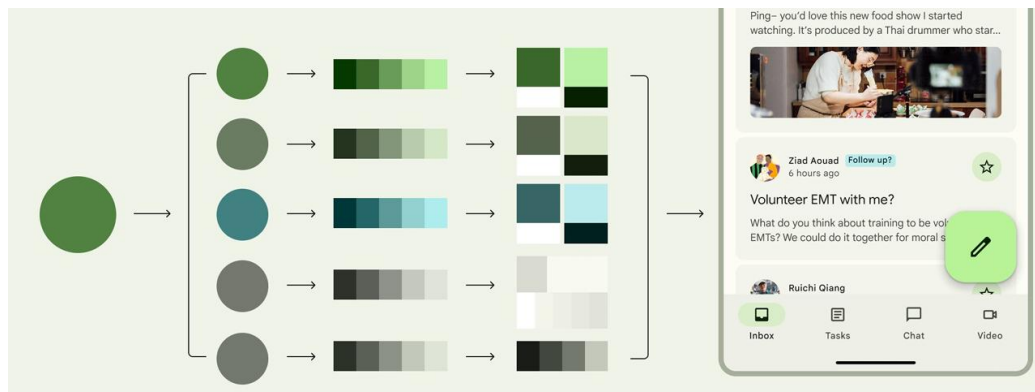
Navigation Bars

Le Navigation Bars contengono da 3 a 5 destinazioni di uguale importanza.

Sono utilizzate per le destinazioni di primo livello che devono essere accessibili da qualsiasi punto dell'app. Contengono icone e etichette di testo opzionali. Non devono essere usate per attività singole, come visualizzare una singola email, preferenze utente o impostazioni.



Colori



Material Design 3 definisce un sistema cromatico che utilizza algoritmi per generare palette coerenti e **accessibili** a partire da un colore di riferimento. Questo sistema è valido anche per iOS. Tale modello di colore si basa su tre componenti principali:

1. **Ruoli dei Colori:** Etichette che descrivono come e dove usare i colori nell'interfaccia.
2. **Assegnazione Dinamica:** Ogni elemento visivo è collegato a un ruolo, che ne determina il colore.
3. **Algoritmo della Palette:** Genera tonalità e livelli di luminosità partendo da un seed color.

Ruoli dei Colori (Color Roles)

Un ruolo di colore è un'etichetta che associa un elemento visivo a uno scopo specifico, descrivendone l'uso e non il colore specifico.

- **Primary:** Elementi interattivi principali (es. pulsanti).
- **Secondary:** Accenti visivi e complementi.
- **Tertiary:** Colori opzionali per variazioni stilistiche.
- **Error:** Stati di errore o avvisi.
- **Background:** Sfondo generale dell'interfaccia.
- **Surface:** Superfici come card o finestre.
- **Outline:** Bordature o contorni degli elementi.
- **On- roles (es. On Primary, On Background):** Testi o icone sovrapposti a un colore di sfondo (garantisce contrasto).
- **Container roles (es. Primary Container):** Colore di sfondo di un contenitore che usa il colore principale come accento.

Primary	Secondary	Tertiary	Error
On Primary	On Secondary	On Tertiary	On Error
Primary Container	Secondary Container	Tertiary Container	Error Container
On Primary Container	On Secondary Container	On Tertiary Container	On Error Container

Il colore finale assegnato a un ruolo dipende dalla **modalità del tema** (chiara o scura) e dalle **tonalità calcolate dall'algoritmo**. Ad esempio, un elemento etichettato come "Primary" in un tema scuro riceverà un colore diverso da un tema chiaro.

Algoritmo di Generazione dei Temi

L'algoritmo M3 (definito in Material Color Utilities) segue questi passaggi partendo da un **seed color** (colore di riferimento, spesso del brand)⁹³:

1. **Generazione di 5 Key Color Complementari:** Dal seed color, l'algoritmo genera 5 colori chiave⁹⁴⁹⁴⁹⁴⁹⁴.
 - Primary
 - Secondary
 - Tertiary
 - Neutral
 - Neutral variant
2. **Generazione di Palette di Tonalità:** Per ogni Key Color viene creata una palette di **13 livelli di luminosità (tonalità)**, da Tone 0 (più scuro/nero) a Tone 100 (più chiaro/bianco).
 - Le tonalità chiare sono utilizzate per sfondi in modalità chiara.
 - Le tonalità scure sono utilizzate per testi in modalità scura.
3. **Assegnazione ai Ruoli:** Usando regole di accessibilità (per garantire il giusto contrasto), l'algoritmo assegna i livelli di luminosità della palette ai ruoli di colore.

Esempio per tema chiaro: Il ruolo Primary potrebbe ricevere la tonalità Tone 40 per un pulsante riempito, e il ruolo On Primary riceverebbe Tone 100 (bianco) per il testo, assicurando un contrasto ottimale.

Strumenti di Creazione Temi

La creazione di temi M3 è semplificata dall'uso di strumenti come il plug-in **Material Theme Builder** per **Figma**, in cui il designer inserisce il **seed color** (il colore del brand) ed in automatico viene generata l'intera palette di colori da assegnare ai singoli ruoli.

Utilizzando l'M3 Design Kit (che ha i ruoli di colore assegnati), è possibile aggiornare l'interfaccia selezionando i componenti e scegliendo un tema (chiaro, scuro o alto contrasto).

Modi

GID – Graphical Devices

Prima di affrontare il concetto di "modo", è necessario definire con precisione il vocabolario tecnico relativo all'input fisico.

- **Content (Contenuto):** insieme delle informazioni nel sistema che hanno significato e utilità per l'utente.
- **GID (Graphical Input Device):** dispositivo usato per comunicare una posizione o selezionare un oggetto.
Attenzione però, i GID non includono tutti i dispositivi di input. Pensiamo ad esempio ad una tastiera... Questa NON ha un aspetto grafico, non mostra niente sullo schermo.
Un mouse o un trackpad, invece, rientrano perfettamente in questa categoria, poiché mostrano un cursore sullo schermo che ci informa sulla posizione e sulla selezione (proprio come nella definizione). Anche il touchscreen rientra perfettamente in questa categoria, ha molta componente grafica e ci permette di muoverci e selezionare.
- **GID Button:** il tasto principale del dispositivo di input (es. tasto destro del mouse).

Azioni Elementari

- **Tap:** Pressione e rilascio di un tasto che ritorna automaticamente allo stato originale.
- **Click:** Posizionamento del GID seguito da un tap sul GID button.
- **Double-click:** Due click consecutivi senza spostare il GID tra l'uno e l'altro.
- **Drag / Swipe:** Pressione del GID button, movimento del GID (senza rilasciare il tasto) e rilascio finale in una nuova posizione.

Gesture

Def. Un **gesto** è una sequenza di azioni che, una volta avviata, viene completata dall'utente (senza pause cognitive), come se fosse un'unica azione.

ATTENZIONE! Queste gesture sono completamente diversi da quelle che oggi intendiamo sugli smartphone (come 3 dita verso l'alto per fare lo screenshot o pinch to zoom, etc..)

Esempi:

- Scrivere una parola con la tastiera. Se ad esempio vogliamo scrivere la parola "che" non ci fermiamo alla scrittura delle singole lettere "c", "h", "e", ma vediamo il tutto come un unico "comando mentale".
- il drag and drop. Sebbene sia formato da una serie (abbastanza lunga) di azioni come: premere su oggetto, tenere premuto, navigare fino al punto da raggiungere (continuando a tenere premuto l'oggetto), rilasciare... il drag and drop viene concepito come un unico gesto. Una volta iniziato si arriva alla fine senza accorgersi delle tante azioni coinvolte.

Modi

Def Un'interfaccia si trova in un "modo" se l'interpretazione di un determinato gesto cambia a seconda dello stato del sistema.

Non modale: Il gesto produce sempre lo stesso risultato.

Modale: Lo stesso gesto ha effetti diversi in momenti diversi.

Esempi:

- Il tasto "Invio" può significare "vai a capo", "conferma inserimento" o "seleziona elemento" a seconda del contesto => l'interfaccia è modale rispetto al tasto invio.
- Le freccette (↑ ↓ ← →) possono assumere diverse funzioni a seconda del contesto. Ad esempio in Excel: con le freccette normalmente ci spostiamo tra una cella e l'altra... Ma quando facciamo doppio click su una cella (entriamo in modalità modifica di una cella) le freccette ci permettono di spostarci lettera per lettera nel testo della cella => l'interfaccia è modale rispetto al premere le freccette
- Il tasto + di un forno. Se premo il pulsante "+" quando vedo l'ora, cambio l'ora. Se premo il pulsante "+" quando vedo la temperatura, alzo i gradi => l'interfaccia è modale rispetto al tasto "+"

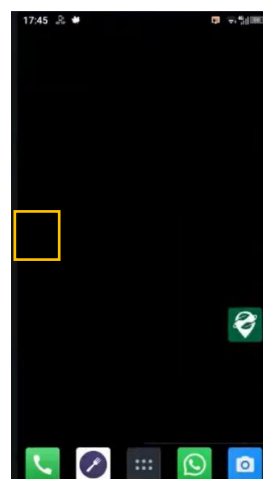
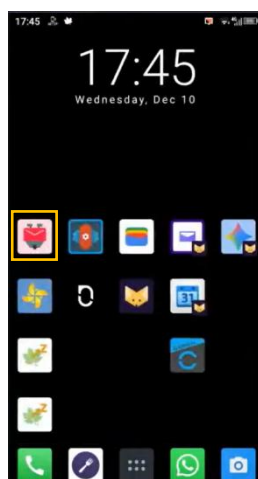
Vantaggi e Svantaggi

La presenza di modi solitamente è sconsigliata, anche se può portare dei vantaggi, soprattutto in termini di risparmio. Nel caso del forno ci permette di avere un unico display per più azioni, o nel caso di excel ci permette di assegnare ad un solo tasto più azioni. Ma rischiamo di creare confusione negli utenti...

Inoltre la presenza di modi costringe l'utente a verificare lo stato del sistema prima di agire:

1. **Caps Lock (Blocco Maiuscole):** È l'esempio classico di mode error. Spesso si inizia a scrivere guardando la tastiera o il testo da copiare, e ci si accorge solo dopo aver scritto diverse parole che il blocco maiuscole era attivo. Nonostante ci siano indicatori (LED, icone a schermo), l'errore persiste (vedremo più avanti che il problema è legato al Luogo dell'Attenzione).
2. **La Torcia:** Se ha un solo pulsante per accensione/spegnimento e la teniamo in tasca, non possiamo sapere l'effetto della pressione del tasto senza guardarla (verificare lo stato).
3. **La Sveglia Tradizionale:** Se c'è un unico tasto per attivare/disattivare, al buio non sappiamo se premendolo attiveremo o disattiveremo l'allarme.

Tuttavia, nel caso in cui lo stato del sistema è chiaramente visibile si riducono i problemi legati all'utilizzo dei modi. Ad esempio, consideriamo le seguenti schermate:



La pressione della stessa cella, a seconda della schermata che stiamo visualizzando, attiverà due funzionalità diverse:

- **Schermata 1:** Apre le email
- **Schermata 2:** Non apre nulla

Questo è certamente un modo: stessa azione => due risultati diversi. Tuttavia, rendere i modi visivamente distinti (es. icone diverse, colori) aiuta a minimizzare l'errore... ma non li risolve! Affinché questo modo non sia problematico, l'indicatore dello stato deve trovarsi nel **Luogo dell'Attenzione** dell'utente nel momento in cui compie il gesto.

Luogo dell'Attenzione

Def. Il Luogo dell'Attenzione (Locus of Attention) è l'oggetto fisico o l'idea su cui stiamo pensando attivamente e intenzionalmente.

Conseguenza: Se un indicatore di stato (come il LED del Caps Lock) si trova al di fuori del luogo dell'attenzione (che è sul testo che stiamo scrivendo o copiando), l'utente non lo percepisce.

Regola d'oro: Può esserci **un solo** luogo dell'attenzione alla volta.

Expert Users vs Mode Errors

Contrariamente a quanto si pensi, l'essere un utente esperto **non protegge** dagli errori di modo. I principianti sbagliano per inesperienza. Gli esperti sbagliano perché hanno acquisito abitudini e agiscono in automatismo, non controllando lo stato del sistema.

Possibili Soluzioni

Secondo Donald Norman i mode errors avvengono per un'errata classificazione della situazione. Per minimizzarli:

1. **Eliminare i modi** (soluzione ideale).
2. **Rendere i modi visivamente distinti in maniera inequivocabile** (sempre nel luogo dell'utente, così siamo sicuri che l'utente non possa non vederli). Tuttavia non è sempre possibile.
3. **Assicurarsi che i comandi tra modi diversi non si sovrappongano** (così se sbaglio modo, il comando non fa nulla o non fa danni).

Modi Temporanei e Quasi-Modi

Esistono implementazioni dei modi meno dannose:

- **Modo Temporaneo:** Svanisce dopo un singolo uso (es. il "pennello copia formato" di Word se cliccato una volta sola).
- **Quasi-modo (Consigliato):** Un modo che rimane attivo solo finché l'utente mantiene fisicamente un controllo (es. tenere premuto SHIFT per le maiuscole o CTRL). È efficace perché la tensione muscolare ricorda costantemente all'utente che si trova in uno stato alternativo (cinestesia).

Paradigmi di Interazione

Questa è una distinzione fondamentale nel design delle interfacce.

Verb-Noun (Azione-Oggetto)

Si seleziona prima l'azione (strumento) e poi l'oggetto su cui applicarla.

PROBLEMA:

1. Sposto l'attenzione sull'azione, poi sull'oggetto. Se mi distraigo, dimentico che lo strumento è attivo e rischio errori (es. volevo selezionare ma dipingo).

Esempio 1: Disegnare su Paint: Seleziono lo strumento "Pennello" -> Cliccare sul disegno.

Esempio 2: Premo "Modifica" -> Seleziono le foto -> azione esplicita per uscire ("Fine").

Noun-Verb (Oggetto-Azione)

Si seleziona prima l'oggetto (che è già nel luogo dell'attenzione) e poi si applica l'azione.

VANTAGGI:

1. Si lavora sull'oggetto mentre è nel luogo dell'attenzione.
2. Richiede meno spostamenti di attenzione.
3. Non richiede comandi di uscita ("Esc" o "Cancel") per interrompere, garantendo maggiore reversibilità.

Esempio 1: Seleziono il testo, poi scelgo il font dal menu.

Esempio 2: Guardo l'oggetto (es. email) -> Long press o Swipe -> Appare il comando "Archivia" lì vicino.